M04 — Prácticas: integración con APIs y seguridad básica

Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: PRÁCTICAS

AUTOR: JAIME GALLO

Índice del módulo

- P4.1 — Consumo HTTP desde Blazor
- P4.2 — Refactorización a servicio tipado
- P4.3 — Construcción de una API REST desde cero
- P4.4 — Seguridad básica: sesión y autorización

Cómo seguir las prácticas

Este manual distingue el **recorrido obligatorio**, que se construye durante las seis horas del módulo, de los **talleres de ampliación**, que sirven para profundizar, repetir y diagnosticar fuera del tiempo presencial. Ningún taller adicional es necesario para que funcione el siguiente punto. Los proyectos de referencia permiten comparar resultados, pero no sustituyen los pasos de construcción escritos aquí.

Itinerario presencial de seis horas

Punto	Explicación	Construcción obligatoria
4.1. HTTP y DTO, GET/POST desde Blazor	25 min	75 min
4.2. Interfaz, DI y cliente HTTP tipado	20 min	50 min
4.3. API mínima creada desde cero	30 min	75 min
4.4. Cookie de demostración, roles y autorización	15 min	40 min
Repaso, dudas y recuperación de incidencias	**30 min**	**—**
Total		**360 minutos (6 h)**

El tiempo de explicación suma 90 minutos y el de construcción obligatoria 240; el cierre ocupa 30 minutos. Las actividades adicionales que figuran después de los recorridos esenciales no se incorporan a ese cómputo presencial.

Cómo reconocer una ampliación

Los talleres de diagnóstico HTTP, el servicio simulado, el archivo de peticiones `.http`, el reto de búsqueda y el análisis extendido de sesiones se presentan como **ampliaciones**. Su objetivo es proporcionar material de referencia y autonomía sin convertir una jornada de seis horas en una lista imposible de ejercicios obligatorios.

Antes de comenzar un nuevo punto, confirma únicamente el resultado obligatorio del anterior: en 4.1 la consulta y el alta remotas; en 4.2 la misma pantalla consumiendo un servicio tipado; en 4.3 la API construida desde cero y compatible; en 4.4 una sesión de demostración que protege un recurso del servidor.

P4.1 — Primer consumo HTTP desde Blazor

Resultado que construiremos

Partimos de la aplicación `M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor`. Crearemos **una copia independiente** en `M04_APIS_SEGURIDAD/PROYECTO/4.1/CursoWeb.Blazor`, sin editar los ejercicios originales de M03. La página `/productos` de M03 se conservará para comparar el catálogo local con el remoto. Incorporaremos una nueva página `/catalogo-remoto` que leerá productos desde una pequeña API; después permitirá crearlos y mostrará los errores sin cerrar la interfaz.

Los ejemplos razonados de HTTP, DTO, JSON y `HttpClient` están explicados en teoría 4.1. Esta práctica se centra en **construir y observar** el comportamiento.

Preparación autónoma del entorno en Visual Studio y PowerShell

Los ejercicios de este módulo utilizan .NET 8. Las aplicaciones necesarias están dentro del repositorio: partimos de la aplicación Blazor del módulo 3 y utilizamos una API auxiliar ya operativa durante 4.1 y 4.2. Después el alumno construirá su propia API en 4.3. El hecho de que ambos proyectos compartan un repositorio no significa que sean un mismo proceso.

Preparación E1. Comprobar el SDK y compilar los proyectos de referencia

Desde PowerShell, situado en la raíz del repositorio, ejecuta:

```
dotnet --list-sdks
dotnet build .\M04_APIS_SEGURIDAD\PROYECTO\4.
1\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj
dotnet build .\M04_APIS_SEGURIDAD\PROYECTO\4.1\CursoWeb.Blazor\CursoWeb.Blazor.csproj
```

Debe existir un SDK 8.x. Una compilación correcta confirma que las dependencias y tipos del proyecto pueden resolverse, pero todavía no demuestra que la API responda ni que la página Blazor obtenga información. Esas comprobaciones se harán después de iniciar los procesos.

Si aparece un error de compilación, resuélvelo antes de modificar la aplicación. Comprueba que trabajas con la ruta correcta y no con una copia incompleta del proyecto, que el SDK apropiado está instalado y que no has perdido archivos al crear tu espacio de trabajo.

Preparación E2. Crear una copia de trabajo sin sobrescribir M03

El proyecto M03 es una referencia que debe permanecer intacta. Desde la raíz del repositorio, prepara una carpeta local de trabajo:

```
New-Item -ItemType Directory -Force .work\M04 | Out-Null
Copy-Item -Path .\M03_BLAZOR\PROYECTO\3.6\CursoWeb.Blazor -Destination .
\.work\M04\CursoWeb.Blazor -Recurse
```

Realiza la copia solamente cuando el destino esté vacío. Si ya existe un trabajo anterior en esa ruta, utiliza una carpeta nueva; no lo sobrescribas. La copia será el punto de partida de los cambios que el alumno escribe durante P4.1.

La API suministrada puede arrancarse desde su ruta versionada. No necesitas copiarla ni modificarla para completar los dos primeros puntos. Cuando llegue P4.3, crearás una aplicación servidor independiente y podrás comparar sus archivos con los del recurso suministrado.

Preparación E2 bis. Activar el estado final de M03 antes de continuar

El proyecto de referencia de M03 conserva intencionadamente comentado el método de carga del paso 3.6.2 para que el alumno pueda activarlo. **La copia de trabajo de M04 debe partir del estado de M03 con ese paso ya realizado.** En la copia —nunca en la baseline de M03— abre `Components/Pages/Productos.razor`, localiza el bloque `PASO 3.6.2` y retira solamente sus delimitadores de comentario. El método resultante será:

```
protected override async Task OnInitializedAsync()
{
    estado = "Cargando productos...";
    productos = (await Catalogo.ObtenerAsync()).ToList();
    cargando = false;
    estado = "Lista preparada";
}
```

Abre `/productos` y confirma que termina la carga y se muestran los productos locales. **No actives simultáneamente la variante `RETO INTEGRADOR 3.6.7`**, porque introduce otra implementación de `OnInitializedAsync` que se utiliza como alternativa. Este paso garantiza que las prácticas M04 no arranquen de una pantalla M03 permanentemente pendiente.

Los proyectos de referencia acumulativos `PROYECTO/4.1` a `PROYECTO/4.4` incorporan ya la activación anterior. Se trata de una precondición de continuidad, no de una nueva funcionalidad HTTP.

Preparación E3. Mantener abiertas dos terminales

En la primera terminal inicia la API:

```
dotnet run --project .\M04_APIS_SEGURIDAD\PROYECTO\4.
1\CursoWeb.ProductosApi\CursoWeb.ProductosApi.csproj --urls http://localhost:5081
```

No cierres esa terminal. Comprueba mediante un navegador que http://localhost:5081/api/productos devuelve el catálogo inicial. Si el puerto está ocupado, comprueba si otra instancia anterior sigue ejecutándose antes de cambiar números en todos los proyectos.

En una segunda terminal inicia la aplicación Blazor de referencia:

```
dotnet run --project .\M04_APIS_SEGURIDAD\PROYECTO\4.1\CursoWeb.Blazor\CursoWeb.Blazor.csproj
--urls http://localhost:5091
```

Ahora Blazor utiliza el puerto 5091 y la API el 5081. Abre http://localhost:5091/catalogo-remoto. El navegador solicita la página Blazor; el servidor que aloja el componente utiliza HttpClient para obtener datos de la API. Esa distinción de procesos es el fundamento de este módulo.

Cuando hayas construido los cambios en tu copia de .work, sustituye en el comando la ruta del proyecto Blazor de referencia por la de tu copia. No necesitas modificar el código de la API para que funcione el cliente.

Preparación E4. Distinguir la consulta directa de la interfaz

Con la API ejecutándose, utiliza una terminal para enviar una petición directa:

```
curl.exe -i http://localhost:5081/api/productos
```

Debes observar 200 y una lista JSON. Esta solicitud no atraviesa ni el componente Razor ni el servicio HTTP de Blazor. Es una evidencia independiente de que el servidor API funciona.

Después detén únicamente la API y repite la petición. No recibirás un 404 de ese servidor: no existe un proceso escuchando para producir su respuesta HTTP. Reinicia la API y comprueba que vuelve a responder antes de continuar.

Si desde Blazor la consulta falla aunque el GET directo funcione, investiga la configuración del cliente, la ruta relativa y los DTO. Si el GET directo también falla, comienza el diagnóstico por el proceso API y el puerto, no por el HTML de la pantalla.

Preparación E5. Ejecutar desde Visual Studio

Puedes abrir los proyectos en ventanas independientes de Visual Studio o incorporarlos a una solución de trabajo. La elección de ventana no cambia el hecho de que son dos aplicaciones diferentes. Identifica siempre qué proyecto inicia el proceso API y qué proyecto inicia Blazor.

Cuando practiques errores de conexión, detén solo la API. Cuando quieras observar una sesión limpia de Blazor, reinicia únicamente su proceso. Cuando quieras restaurar la colección inicial de productos, reinicia la API, porque es allí donde reside la memoria de productos.

No hay que crear un controlador MVC ni una tercera pantalla para que la API responda. La plantilla web mínima que reconstruirás en 4.3 es suficiente para publicar el pequeño contrato HTTP del laboratorio.

Preparación E6. Entender la diferencia entre build y run

dotnet build compila. dotnet run inicia un proceso. Un proyecto puede compilar correctamente y, sin embargo, fallar al abrir un puerto ocupado o al intentar alcanzar un servidor distinto que está apagado. También puede iniciarse una API que todavía no publica todos los endpoints esperados. Por eso las prácticas combinan compilación, consulta HTTP directa y resultado visible en Blazor.

No confundas un error de resolución de dependencias con un error de conexión. En 4.2 comprobaremos expresamente que el componente necesita tener registrado IProductosApiService; en 4.1 todavía usa el cliente con nombre. Son estados de construcción distintos y deben conservarse para entender la evolución.

Estado de partida para comenzar P4.1

Antes de pasar al primer paso debes poder identificar la carpeta de M03 que sirve como base, el lugar donde escribes tu copia, la terminal donde corre la API y la terminal donde corre Blazor. También debes haber comprobado el JSON del catálogo fuera de Blazor.

Esta preparación no adelanta la implementación de 4.2, la creación desde cero de 4.3 ni el acceso de demostración de 4.4. Su objetivo es que los cambios de cada punto puedan atribuirse a una decisión concreta y no a un entorno que nunca llegó a estar operativo.

Paso 4.1.1. Preparar dos procesos independientes

Proyecto API suministrado: `M04_APIS_SEGURIDAD/PROYECTO/4.1/CursoWeb.ProductosApi`.

Proyecto Blazor inicial: copia de `M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor`.

Abre dos terminales. En la primera, desde la carpeta de la API, ejecuta:

```
dotnet run --urls http://localhost:5081
```

En un navegador, visita `http://localhost:5081/api/productos`. Debe aparecer un array JSON con Cuaderno y Lápiz. Visita `http://localhost:5081/api/productos/999` para provocar un **404**: observa que una API puede contestar correctamente a la petición HTTP e indicar al mismo tiempo que un recurso no existe.

No avances si el servidor no arranca: comprueba que tienes el SDK de .NET 8 y que el puerto 5081 no está ocupado. Este servidor no es una aplicación MVC ni la interfaz de Blazor.

Paso 4.1.2. Crear los DTO en el cliente

En `CursoWeb.Blazor/Models/` añade `ProductoDto.cs`:

```
namespace CursoWeb.Blazor.Models;
public sealed record ProductoDto(int Id, string Nombre, decimal Precio);
```

Añade `CrearProductoDto.cs`:

```
namespace CursoWeb.Blazor.Models;
public sealed record CrearProductoDto(string Nombre, decimal Precio);
```

Compara estos contratos con el `Producto` de M03. No sustituyas el modelo de M03: sus tarjetas, filtros y edición local seguirán compilando con su propio tipo. Aquí usamos nuevos DTO para delimitar los datos que llegan por la red y los que enviamos al crear un producto.

Paso 4.1.3. Configurar el cliente HTTP

En `CursoWeb.Blazor/Program.cs`, **mantén** el registro de componentes y el servicio `IProductosMemoriaService` de M03. Añade:

```
builder.Services.AddHttpClient("ProductosApi", cliente =>
{
    cliente.BaseAddress = new Uri(
        builder.Configuration["ProductosApi:BaseUrl"]
        ?? "http://localhost:5081/");
    cliente.Timeout = TimeSpan.FromSeconds(10);
});
```

En el `appsettings.json` de Blazor declara una sección `ProductosApi` con `BaseUrl` igual a `http://localhost:5081/`. Si el archivo no existe en tu copia de M03, créalo con un objeto JSON que contenga esa sección.

El registro crea un cliente HTTP **con nombre**. No implementamos todavía la interfaz `IProductosApiService`; la construiremos en el ejercicio 4.2, cuando ya conozcamos el circuito que debe encapsular.

Paso 4.1.4. Construir una pantalla de consulta remota

Crea `Components/Pages/CatalogoRemoto.razor`. Incluye una directiva `@page "/catalogo-remoto"`, el modo `InteractiveServer`, la inyección de `IHttpClientFactory` y una colección `List<ProductoDto>`.

En `OnInitializedAsync`, usa `Clientes.CreateClient("ProductosApi")` y `GetFromJsonAsync<List<ProductoDto>>("api/productos")`. Presenta tres estados separados: «Cargando…», listado y error.

El **código íntegro de esta primera versión** figura inmediatamente a continuación en estas mismas prácticas. Escríbelo y comprueba sus resultados; comprueba que al navegar a `/catalogo-remoto` se ven los mismos dos productos que aparecen en el JSON del navegador.

Una observación importante: el GET se realiza desde el proceso de Blazor en Interactive Server, no desde el navegador; no necesitamos añadir una política CORS ficticia a la API de este laboratorio.

Código íntegro de consulta remota (paso 4.1.4)

El siguiente archivo es el **estado de partida compilable** de `Components/Pages/CatalogoRemoto.razor` antes de añadir POST. Escríbelo después de registrar el cliente HTTP y los DTO; así puedes comprobar GET de forma independiente sin anticipar el resultado del paso siguiente.

```
@page "/catalogo-remoto"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@using System.Net.Http.Json
@inject IHttpClientFactory Clientes

<h1>Catálogo remoto</h1>
@if (cargando)
{
    <p role="status">Cargando catálogo...</p>
}
else if (error is not null)
{
    <p role="alert">@error</p>
}
else
{
    <ul>
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre — @producto.Precio</li>
        }
    </ul>
}

@code {
    private List<ProductoDto> productos = [];
    private bool cargando = true;
    private string? error;

    protected override async Task OnInitializedAsync()
    {
        try
        {
            var cliente = Clientes.CreateClient("ProductosApi");
            productos = await cliente.GetFromJsonAsync<List<ProductoDto>>(
                "api/productos") ?? [];
        }
```

```
        catch (HttpRequestException)
        {
            error = "No se pudo obtener el catálogo remoto.";
        }
        catch (TaskCanceledException)
        {
            error = "La petición superó el tiempo disponible.";
        }
        catch (System.Text.Json.JsonException)
        {
            error = "La API no devolvió un catálogo JSON válido.";
        }
        finally
        {
            cargando = false;
        }
    }
}
```

La directiva `@rendermode` activa el circuito interactivo, aunque la consulta inicial se efectúe desde el servidor Blazor. Antes de pasar a POST, arranca primero la API y luego Blazor. Comprueba que aparecen Cuaderno y Lápiz. Detén la API y recarga la página: debe aparecer el aviso de fallo y nunca un catálogo vacío presentado como éxito.

Paso 4.1.5. Añadir un formulario de alta

A la pantalla anterior incorpora dos campos, `Nombre` y `Precio`, y un botón «Crear en API». El botón llama a un método `async Task`:

```
private async Task CrearAsync()
{
    if (string.IsNullOrWhiteSpace(nombreNuevo) || precioNuevo <= 0)
    {
        mensaje = "Indica un nombre y un precio mayor que cero.";
        return;
    }

    enviando = true;
    mensaje = null;
    try
    {
        var cliente = Clientes.CreateClient("ProductosApi");
        using var respuesta = await cliente.PostAsJsonAsync(
            "api/productos",
            new CrearProductoDto(nombreNuevo.Trim(), precioNuevo));
        respuesta.EnsureSuccessStatusCode();

        var creado = await respuesta.Content
            .ReadFromJsonAsync<ProductoDto>();
        if (creado is null)
            throw new InvalidOperationException("Alta sin producto de respuesta.");

        productos.Add(creado);
        nombreNuevo = string.Empty;
        precioNuevo = 0;
        mensaje = $"Producto creado: {creado.Nombre} (id {creado.Id}).";
    }
    catch (HttpRequestException)
    {
        mensaje = "La API no pudo completar el alta.";
    }
```

```
    catch (TaskCanceledException)
    {
        mensaje = "Se agotó el tiempo al crear el producto.";
    }
    catch (System.Text.Json.JsonException)
    {
        mensaje = "La respuesta de alta no contiene JSON válido.";
    }
    catch (InvalidOperationException)
    {
        mensaje = "La respuesta de alta está incompleta.";
    }
    finally
    {
        enviando = false;
    }
}
```

La página necesita los campos `private string nombreNuevo = string.Empty;`, `private decimal precioNuevo;`, `private bool enviando;` y `private string? mensaje;`. Deshabilita el botón mientras `enviando` es verdadero. Usa `@bind` en los inputs y `@onclick="CrearAsync"` en el botón. Añade `@using System.Net.Http.Json` si no existe una importación global equivalente.

Crea «Regla» con precio 1,75. Debe aparecer una línea nueva con el identificador asignado por la API. Recarga la página: el producto sigue presente mientras la API continúa ejecutándose. Reinicia la API: la colección vuelve a sus datos iniciales, porque todavía no existe persistencia.

Paso 4.1.6. Forzar tres tipos distintos de fallo

Datos rechazados: envía directamente a la API, desde la terminal, una petición POST con precio cero:

```
curl.exe -i -X POST http://localhost:5081/api/productos -H "Content-Type: application/json" -d
"{\"nombre\":\"Regla\",\"precio\":0}"
```

Si tu terminal altera las comillas de JSON, usa un archivo `producto-invalido.json` y `--data-binary "@producto-invalido.json"`. Debes observar **400 Bad Request**. La interfaz de Blazor, por su parte, impide el envío de valores inválidos mediante una comprobación previa.

Ruta inexistente: solicita `http://localhost:5081/api/productos/999`; observa **404 Not Found**.

Servicio no disponible: detén el proceso de la API, recarga `/catalogo-remoto` y observa el aviso de comunicación. Vuelve a iniciar la API y recarga la página. No interpretes una colección vacía como señal suficiente de error: tenemos estados distintos para éxito y fallo.

Paso 4.1.7. Reto: distinguir el rechazo HTTP del fallo de red

La versión de consulta basada en `GetFromJsonAsync` ya detecta excepciones, pero no presenta el código HTTP de un fallo. Sustituye **solo la lectura GET** por un `GetAsync` explícito:

```
using var respuesta = await cliente.GetAsync("api/productos");
if (!respuesta.IsSuccessStatusCode)
{
    error = $"La API respondió {(int)respuesta.StatusCode}.";
    return;
}
```

```
productos = await respuesta.Content
    .ReadFromJsonAsync<List<ProductoDto>>() ?? [];
```

Mantén el `finally` que desactiva el indicador de carga. Puedes cambiar temporalmente la ruta por `api/no-existe` y comparar «La API respondió 404» con el error de conexión que aparece al apagar el servidor. Restablece la ruta real al terminar el reto.

Taller de consolidación P4.1: del GET inicial al archivo final

Una vez que la consulta funciona, necesitamos convertirla en una pantalla que permita crear productos sin perder las garantías anteriores. No copies el archivo final al principio de la práctica: compáralo con el estado exclusivamente GET que ya has compilado. Localiza los nuevos elementos, identifica su propósito y escríbelos en el mismo componente.

Primero aparecen los inputs de Nombre y Precio. En Blazor los valores se enlazan con variables de estado mediante @bind. Estos campos no realizan todavía una llamada HTTP: solamente cambian el estado del componente.

En segundo lugar aparece el botón Crear en API. Su evento @onclick invoca un método Task, y la propiedad disabled observa el estado enviando. La validación inicial comprueba nombre no vacío y precio mayor que cero. Es una ayuda al usuario: el servidor volverá a validar independientemente esos datos.

En tercer lugar aparece CrearAsync. Esta operación hace POST y lee la respuesta. Si la API confirma la creación, añade a la colección el objeto devuelto, que ya contiene el identificador asignado. Solo entonces limpia los dos inputs. Si ocurre un error, presenta un mensaje y conserva los valores para que el alumno pueda corregirlos o intentarlo de nuevo cuando la API esté disponible.

El siguiente archivo reúne el estado final de P4.1. Se muestra íntegro para que el alumno pueda comparar y localizar cada diferencia; no sustituye los pasos previos, porque se trata de comprender cómo se ha llegado a él.

```
@page "/catalogo-remoto"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@using System.Net.Http.Json
@inject IHttpClientFactory Clientes
<PageTitle>Productos desde API</PageTitle>

<h1>Catálogo remoto · P4.1</h1>
<p>Esta página consume una API HTTP. El catálogo local de M03 sigue disponible en /productos.</
p>
@if (cargando)
{
    <p id="estado-api" role="status">Cargando catálogo remoto...</p>
}
else if (error is not null)
{
    <p id="error-api" role="alert">@error</p>
}
else
{
    <p id="estado-api">Catálogo remoto disponible</p>
    <ul id="productos-api">
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
        }
    </ul>
```

```
}
<section>
    <h2>Crear producto</h2>
    <label>Nombre: <input id="nombre-api" @bind="nombreNuevo" /></label>
    <label>Precio: <input id="precio-api" type="number" min="0" step="0.01"
@bind="precioNuevo" /></label>
    <button id="crear-api" disabled="@enviando" @onclick="CrearAsync">Crear en API</button>
    @if (mensaje is not null)
    {
        <p id="resultado-api" role="status">@mensaje</p>
    }
</section>

@code {
    private List<ProductoDto> productos = [];
    private bool cargando = true;
    private string? error;
    private string nombreNuevo = string.Empty;
    private decimal precioNuevo;
    private bool enviando;
    private string? mensaje;

    protected override async Task OnInitializedAsync()
    {
        try
        {
            var cliente = Clientes.CreateClient("ProductosApi");
            productos = await cliente.GetFromJsonAsync<List<ProductoDto>>("api/productos") ??
[];
        }
        catch (HttpRequestException)
        {
            error = "No se pudo obtener el catálogo remoto.";
        }
        catch (TaskCanceledException)
        {
            error = "La petición superó el tiempo disponible.";
        }
        catch (System.Text.Json.JsonException)
        {
            error = "La API no devolvió un catálogo JSON válido.";
        }
        finally
        {
            cargando = false;
        }
    }

    private async Task CrearAsync()
    {
        if (string.IsNullOrWhiteSpace(nombreNuevo) || precioNuevo <= 0)
        {
            mensaje = "Indica un nombre y un precio mayor que cero.";
            return;
        }

        enviando = true;
        mensaje = null;
        try
        {
            var cliente = Clientes.CreateClient("ProductosApi");
            using var respuesta = await cliente.PostAsJsonAsync(
                "api/productos", new CrearProductoDto(nombreNuevo.Trim(), precioNuevo));
            respuesta.EnsureSuccessStatusCode();
            var creado = await respuesta.Content.ReadFromJsonAsync<ProductoDto>()
```

```
                ?? throw new InvalidOperationException("Alta sin producto de respuesta.");
            productos.Add(creado);
            nombreNuevo = string.Empty;
            precioNuevo = 0;
            mensaje = $"Producto creado: {creado.Nombre} (id {creado.Id}).";
        }
        catch (HttpRequestException)
        {
            mensaje = "La API no pudo completar el alta.";
        }
        catch (TaskCanceledException)
        {
            mensaje = "Se agotó el tiempo al crear el producto.";
        }
        catch (System.Text.Json.JsonException)
        {
            mensaje = "La respuesta de alta no contiene JSON válido.";
        }
        catch (InvalidOperationException)
        {
            mensaje = "La respuesta de alta está incompleta.";
        }
        finally
        {
            enviando = false;
        }
    }

    /* RETO P4.1: reemplazar temporalmente la lectura GET de OnInitializedAsync.
    using var respuesta = await cliente.GetAsync("api/productos");
    if (!respuesta.IsSuccessStatusCode)
    {
        error = $"La API respondió {(int)respuesta.StatusCode}.";
        return;
    }
    productos = await respuesta.Content.ReadFromJsonAsync<List<ProductoDto>>() ?? [];
    FIN RETO P4.1 */
}
```

Ejecución A: consulta satisfactoria

Arranca la API antes que el cliente Blazor. Comprueba su URL /api/productos directamente y anota cuántos productos devuelve. A continuación abre /catalogo-remoto y verifica que la lista visual corresponde con el JSON observado. Si ambos procesos funcionan pero no aparece el catálogo, inspecciona la dirección base en appsettings.json y la ruta relativa del cliente.

No escribas todavía un producto nuevo. La finalidad de esta primera comprobación es demostrar que el sistema puede consultar datos remotos. Es importante realizarla antes de modificar el estado por POST, porque así dispones de una referencia conocida.

Ejecución B: creación confirmada

Escribe Regla y el precio decimal 1,75 utilizando el formato admitido por el campo de la interfaz. Pulsa Crear en API una sola vez y observa el mensaje de confirmación que incluye el nombre y el ID. Verifica que la lista ahora contiene una entrada nueva. Recarga la página: el nuevo recurso debe seguir presente porque reside en el proceso API y este no se ha reiniciado.

Si el formulario se limpia pero no aparece el producto, revisa en qué instante borras los valores y cuándo añades el DTO devuelto por el servidor. El orden correcto es esperar el resultado,

incorporar el recurso y entonces limpiar. No es correcto confundir «se pulsó el botón» con «el servidor confirmó el alta».

Ejecución C: rechazo por reglas del servidor

El formulario no envía un precio cero, porque realiza una validación previa. Para demostrar que la API mantiene sus propias reglas, prepara un archivo JSON con nombre Regla y precio 0 y envíalo directamente por HTTP desde la terminal. Debes observar 400. Después realiza GET para comprobar que no se ha añadido una entrada nueva.

No hace falta eliminar la validación de Blazor para provocar la situación: en este ejercicio estamos estudiando precisamente la diferencia entre una ayuda visual y una regla del servidor. Si el POST inválido se aceptara, la API tendría un defecto independiente de la interfaz.

Ejecución D: fallo de comunicación

Con ambos procesos operativos, abre la pantalla y comprueba que se ve el catálogo. Ahora detén únicamente la API, sin detener Blazor, y recarga el componente. Debe aparecer un mensaje de comunicación o petición agotada. El hecho de que la página Blazor siga pudiendo representarse mientras falla la API demuestra que son procesos diferentes.

Vuelve a iniciar la API y recarga. Si recupera el listado inicial, has validado el camino de ida y vuelta del cliente. Si el listado contiene todavía productos creados antes de un reinicio completo del servidor, revisa si realmente has detenido el proceso API correcto.

Ejecución E: ruta incorrecta frente a servidor apagado

Con la API encendida, cambia temporalmente la ruta de consulta desde api/productos a api/no-existe. Si utilizas GetAsync y compruebas StatusCode, verás un 404 explícito. Restablece la ruta original antes de continuar. Este resultado no debe confundirse con una conexión rechazada: en el primer caso el servidor existe y responde; en el segundo no recibimos una respuesta HTTP del servidor objetivo.

Ejecución F: contrato JSON incompatible

Como experimento razonado, imagina que el servidor respondiera con el objeto de resumen en la ruta de listado. Ese objeto puede ser JSON correcto, pero la pantalla espera List. Predice qué parte del programa fallaría: no se trata de un problema de @bind ni del botón de crear, sino de deserialización del contrato. La práctica no requiere romper la API de manera permanente para entenderlo.

Hoja de resultados esperados P4.1

Acción	Resultado del servidor	Resultado visible	Interpretación
GET inicial	200, dos productos	Dos entradas	Consulta remota correcta
POST Regla 1,75	201, producto con ID	Tercera entrada y mensaje	Alta confirmada
POST de precio 0 desde HTTP	400	Sin nuevo producto	Regla del servidor
GET del ID 999	404	Respuesta no encontrada	Recurso inexistente
API detenida	Sin respuesta válida	Aviso de conectividad	Límite de proceso

Acción	Resultado del servidor	Resultado visible	Interpretación
API reiniciada	200, estado inicial	Dos entradas	Memoria no persistente

Al cerrar P4.1, el alumno debe poder explicar qué parte del flujo se ejecuta en el proceso Blazor y cuál se ejecuta en el proceso API. Esa explicación es la base del refactor de P4.2.

Ampliación P4.1 — Taller complementario: comparar tres situaciones desde una misma página

Ya hemos creado el catálogo remoto y hemos aprendido a mostrar errores. Ahora construiremos una **pantalla temporal de diagnóstico HTTP**, sin sustituir la aplicación principal. El objetivo es observar de forma diferenciada una respuesta correcta, un rechazo 404 y un fallo de conectividad, manteniendo la misma configuración de cliente.

Esta variante pertenece exclusivamente al estado 4.1, donde el componente todavía utiliza el cliente con nombre ProductosApi. No se arrastra automáticamente a 4.2, porque allí sustituiremos el acceso directo a HttpClient por el servicio tipado. Podemos conservarla como referencia para explicar por qué queremos encapsular las rutas HTTP.

Paso D1. Preparar un componente separado

En Components/Pages crea DiagnosticoHttp.razor con la ruta /diagnostico-http. Declara el modo de renderizado InteractiveServer y utiliza IHttpClientFactory como en P4.1. Este componente no necesita un formulario POST: su función es consultar un recurso y presentar tanto el código recibido como cualquier error de transporte.

No modifiques CatalogoRemoto.razor para esta variante. Mantener las dos páginas permite comparar el comportamiento habitual del catálogo con una herramienta de diagnóstico concebida para el aprendizaje. El archivo completo que debe crear el alumno aparece a continuación.

```
@page "/diagnostico-http"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@using System.Net.Http.Json
@inject IHttpClientFactory Clientes

<h1>Diagnóstico HTTP · variante P4.1</h1>
<p>Esta pantalla compara un GET válido, una ruta 404 y un fallo de conexión.</p>
<div>
    <button id="diagnostico-correcto" disabled="@consultando"
            @onclick='() => ConsultarAsync("api/productos")'>GET catálogo</button>
    <button id="diagnostico-404" disabled="@consultando"
            @onclick='() => ConsultarAsync("api/no-existe")'>GET ruta inexistente</button>
</div>
@if (consultando)
{
    <p role="status">Consultando la API...</p>
}
<p id="diagnostico-resultado" role="status">@resultado</p>
@if (productos.Count > 0)
{
    <ul>
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
        }
    </ul>
}
```

```
@code {
    private bool consultando;
    private string resultado = "Selecciona una petición.";
    private List<ProductoDto> productos = [];

    private async Task ConsultarAsync(string ruta)
    {
        consultando = true;
        resultado = "Esperando respuesta...";
        productos.Clear();
        try
        {
            var cliente = Clientes.CreateClient("ProductosApi");
            using var respuesta = await cliente.GetAsync(ruta);
            if (!respuesta.IsSuccessStatusCode)
            {
                resultado = $"Respuesta HTTP {(int)respuesta.StatusCode}
({respuesta.ReasonPhrase}).";
                return;
            }
            productos = await respuesta.Content
                .ReadFromJsonAsync<List<ProductoDto>>() ?? [];
            resultado = $"HTTP {(int)respuesta.StatusCode}: {productos.Count} producto(s).";
        }
        catch (HttpRequestException)
        {
            resultado = "No existe una respuesta HTTP válida de la API: comprueba host y
puerto.";
        }
        catch (TaskCanceledException)
        {
            resultado = "La consulta agotó el tiempo configurado.";
        }
        catch (System.Text.Json.JsonException)
        {
            resultado = "El cuerpo JSON no se corresponde con una lista de productos.";
        }
        finally
        {
            consultando = false;
        }
    }
}
```

Paso D2. Observar el estado inicial

Arranca primero la API y después Blazor. Abre /diagnostico-http. Antes de pulsar ningún botón, la pantalla debe mostrar la instrucción Selecciona una petición. Este estado inicial no representa todavía una petición pendiente ni un error: simplemente no hemos lanzado una operación.

Comprueba que los dos botones están disponibles. El primero utiliza api/productos y el segundo api/no-existe. Las dos rutas parten de la misma dirección base configurada mediante el cliente con nombre. Por ello, si la configuración apunta a un puerto incorrecto, ambos intentos encontrarán el mismo problema de conectividad.

Paso D3. Consultar el catálogo válido

Pulsa GET catálogo. Durante el tiempo en que la solicitud está pendiente, consultando es verdadero, se muestra el texto de espera y ambos botones quedan temporalmente deshabilitados.

Al llegar una respuesta correcta, el componente lee la lista de productos y presenta el número de elementos.

El resultado esperado en una API recién iniciada es un estado HTTP 200 y dos objetos, Cuaderno y Lápiz. Al consultar la lista, el contenido JSON se convierte a ProductoDto y los objetos aparecen en la pantalla. Este experimento también permite observar que obtener un 200 no evita tener que comprobar la forma del cuerpo antes de dibujar los datos.

Paso D4. Solicitar una ruta inexistente

Pulsa GET ruta inexistente. La URL llega al mismo servidor operativo, pero el endpoint api/no-existe no está publicado. La propiedad IsSuccessStatusCode será falsa. El componente leerá StatusCode y ReasonPhrase y presentará el resultado sin intentar deserializarlo como una lista de productos.

La diferencia respecto al botón anterior aparece en la respuesta HTTP, no en el mecanismo de comunicación: ambos usan el mismo HttpClient, el mismo host y el mismo puerto. Si la página indica 404, la API ha recibido la solicitud y ha respondido. El error no equivale a que no haya conexión.

Paso D5. Provocar ausencia de respuesta

Deja abierta Blazor y detén el proceso API. Pulsa de nuevo GET catálogo. En esta ocasión el host destinatario ya no puede aceptar la petición. Se lanzará una excepción de comunicación y se mostrará el mensaje de conectividad. No inventamos un código HTTP 404, porque el servidor objetivo no ha enviado una respuesta válida.

Este paso demuestra por qué una aplicación puede seguir mostrando una interfaz interactiva aunque el servidor de datos remoto esté apagado: Blazor y la API son procesos diferentes. También explica por qué un fallo de conexión no debería convertirse silenciosamente en una colección vacía.

Paso D6. Restaurar el estado y comparar tiempos

Vuelve a iniciar la API y pulsa GET catálogo. El funcionamiento debe recuperarse sin modificar el archivo Razor. Si sigue fallando, comprueba que la nueva ejecución escucha en el puerto configurado. No cambies simultáneamente la dirección base y la ruta: primero identifica cuál de las dos no coincide.

Para observar la variable consultando, puedes ejecutar la misma operación mientras el servidor tarde algo más en responder. No es necesario incorporar Thread.Sleep al endpoint: lo importante es entender que durante una espera real el componente mantiene un estado visual y, en finally, lo restaura al terminar. No conviertas una pausa artificial de servidor en parte permanente de la API del alumno.

Paso D7. Revisar la diferencia entre tres clases de fallo

El componente distingue HttpRequestException, TaskCanceledException y JsonException. La primera representa problemas de comunicación o determinados estados HTTP, según qué método se utilice; la segunda puede aparecer cuando se supera el tiempo; la tercera indica que el contenido no se ha podido convertir al tipo esperado.

No son tres nombres intercambiables para un mismo error. El alumno debe explicar qué observación tiene para decidir si revisa el puerto, el tiempo de espera o el contrato JSON. En un

entorno real podrían existir más clases de excepción y políticas de observabilidad, pero aquí nos basta con aprender a no esconderlas bajo un mensaje ambiguo.

P4.1 — Ejercicio de contraste con el componente principal

Abre nuevamente /catalogo-remoto. Allí la interfaz resume la consulta y muestra datos o un error. Después abre /diagnostico-http: esta pantalla expone deliberadamente el código HTTP para la demostración. Ambas utilizan el cliente ProductosApi y los DTO del proyecto, pero están diseñadas con fines de presentación distintos.

Si el alumno terminara copiando toda la lógica de diagnóstico a cada pantalla, habría aumentado la duplicación. Esta observación prepara P4.2: la necesidad de situar la comunicación en una clase de servicio mientras los componentes conservan sus respectivas decisiones de presentación.

P4.1 — Resultados que deben poder explicarse

Acción	API disponible	Respuesta recibida	Mensaje principal
GET catálogo	Sí	200 con lista	HTTP 200, número de productos
GET inexistente	Sí	404	Código HTTP 404
GET catálogo	No	Sin respuesta del servidor	Problema de conectividad
GET catálogo tras reinicio	Sí	200 con lista	Recuperación de consulta
JSON incompatible	Sí	Puede devolver 200	Error de conversión de datos

El alumno debe volver a dejar operativa la API al terminar, porque P4.2 parte de un catálogo remoto funcional. La página de diagnóstico puede mantenerse en el proyecto de 4.1 como material de exploración, sin integrarla en los estados siguientes.

Ampliación autónoma P4.1 — Una lista remota no se actualiza sola: construir «Consultar servidor»

Durante el primer ejercicio el componente obtuvo el catálogo al iniciarse. Es un comportamiento correcto para una primera pantalla, pero no significa que todas las modificaciones que otros clientes realicen en la API aparezcan automáticamente en la lista ya representada. Vamos a demostrar esta diferencia con una página independiente que **solo obtiene datos cuando el alumno pulsa un botón**.

El objetivo no es crear un mecanismo avanzado de sincronización. Queremos observar una instantánea local, una modificación externa y una nueva petición GET que trae los cambios. Esta actividad complementaria no es necesaria para pasar a 4.2.

Paso R1. Preparar los dos procesos y un estado inicial

Inicia la API del ejercicio en 5081. En otra terminal inicia Blazor desde el proyecto de 4.1. Consulta directamente GET /api/productos para confirmar que existen Cuaderno y Lápiz.

No ejecutes un POST todavía. El propósito del primer estado es disponer de una colección de referencia cuyo contenido conozcamos. Si la API conserva productos de intentos anteriores, detén completamente ese proceso y reinícialo antes de comenzar.

Paso R2. Crear una página distinta del catálogo principal

En la copia del proyecto Blazor crea el archivo Components/Pages/ActualizarRemoto.razor. Define la ruta /actualizar-remoto, el modo Interactive Server y la inyección de IHttpClientFactory. Esta página coexistirá con /catalogo-remoto y no modificará la interfaz del ejercicio obligatorio.

La nueva página tendrá un botón Consultar servidor, un mensaje de estado, una lista visual y un aviso de error. La decisión clave es que no consultará automáticamente en OnInitializedAsync: esperará a que el usuario pulse el botón.

Escribe el archivo siguiente por completo y compílalo antes de ejecutar:

```
@page "/actualizar-remoto"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@using System.Net.Http.Json
@inject IHttpClientFactory Clientes

<h1>Actualización explícita de un catálogo remoto</h1>
<p>La lista se obtiene cuando se pulsa Consultar. Otro cliente puede modificar
   el servidor sin actualizar automáticamente esta copia visual.</p>

<button id="consultar-remoto" disabled="@consultando" @onclick="ConsultarAsync">
    @(consultando ? "Consultando..." : "Consultar servidor")
</button>

<p id="estado-consulta" role="status">@estado</p>

@if (productos.Count == 0 && !consultando && consultado && error is null)
{
    <p id="sin-coincidencias">El catálogo remoto está vacío.</p>
}
@if (error is not null)
{
    <p role="alert" id="error-consulta">@error</p>
}
<ul id="lista-consultada">
    @foreach (var producto in productos)
    {
        <li>@producto.Id — @producto.Nombre — @producto.Precio.ToString("0.00") €</li>
    }
</ul>

@code {
    private bool consultado;
    private bool consultando;
    private string estado = "Todavía no se ha consultado el servidor.";
    private string? error;
    private List<ProductoDto> productos = [];

    private async Task ConsultarAsync()
    {
        consultando = true;
        error = null;
        estado = "Esperando GET /api/productos...";

        try
        {
            var cliente = Clientes.CreateClient("ProductosApi");
            var recibidos = await cliente
                .GetFromJsonAsync<List<ProductoDto>>("api/productos");

            productos = recibidos ?? [];
```

```
            estado = $"Consulta correcta: {productos.Count} producto(s).";
        }
        catch (HttpRequestException)
        {
            error = "No se pudo consultar la API. Comprueba el host y el puerto.";
            estado = "Consulta fallida.";
        }
        catch (TaskCanceledException)
        {
            error = "La espera de la consulta ha terminado.";
            estado = "Consulta interrumpida.";
        }
        catch (System.Text.Json.JsonException)
        {
            error = "La API no devolvió una lista JSON compatible.";
            estado = "Contrato incompatible.";
        }
        finally
        {
            consultado = true;
            consultando = false;
        }
    }
}
```

Paso R3. Examinar por qué no se consulta durante la inicialización

Localiza los campos consultado y consultando. El primero indica si se ha ejecutado alguna vez una petición; el segundo señala si hay una operación pendiente en ese momento. No son equivalentes.

Al abrir la página por primera vez, consultado es falso y productos es una lista vacía. Eso no significa todavía que el servidor haya confirmado un catálogo vacío. Por ello mostramos un mensaje que dice que aún no hemos consultado. Si representáramos «No hay productos» desde el primer instante, estaríamos transmitiendo una conclusión que aún no está respaldada por una respuesta HTTP.

Después de una consulta correcta cuyo resultado sea una lista vacía, consultado será verdadero y el mensaje de catálogo vacío tendrá sentido. El alumno debe poder explicar esa diferencia sin recurrir a detalles internos de Razor.

Paso R4. Ejecutar el primer GET mediante el botón

Abre /actualizar-remoto. Comprueba que todavía no aparecen productos ni un mensaje de error. Pulsa Consultar servidor. El método ConsultarAsync se ejecuta, crea un cliente con nombre ProductosApi y realiza GET /api/productos.

Si la API está operativa, la respuesta se deserializa como List. El componente asigna la colección al campo productos y muestra el número de elementos recibidos. En un proceso recién iniciado deben aparecer dos elementos.

Observa la variable consultando. Mientras espera, el botón se deshabilita y su texto cambia. Tras éxito o error, finally devuelve consultando a falso. Si se omitiera ese paso, el usuario podría quedar con el botón inutilizable después de un fallo de comunicación.

Paso R5. Crear un producto desde otro cliente

Deja la página abierta, sin pulsar Consultar. Desde PowerShell o el archivo ProductosApi.http envía un POST válido con nombre Regla y precio 1,75.

Comprueba mediante una consulta HTTP directa que la API contiene ahora tres productos. No recargues todavía el navegador y no pulses el botón de la página Blazor.

Observa el listado que permanece visible en /actualizar-remoto. Seguirá mostrando el estado de la última consulta que realizó, es decir, dos productos. Eso no es necesariamente un defecto: la pantalla no ha solicitado todavía otra instantánea al servidor.

Paso R6. Consultar de nuevo y comparar ambas listas

Pulsa Consultar servidor. La lista visual debe mostrar ahora tres productos. El nuevo DTO contiene el ID real de Regla, asignado por la API. La variable de estado pasa a reflejar el nuevo número de elementos.

Esta operación no cambia los datos del servidor: es un GET. La diferencia visual aparece porque la interfaz ha reemplazado su colección local con una representación más reciente. El alumno puede describir la secuencia en términos de dos momentos: el servidor fue modificado por POST desde otro cliente y Blazor observó el cambio mediante un nuevo GET.

Paso R7. Demostrar que un segundo cliente puede consultar sin usar Blazor

Abre una nueva terminal y ejecuta GET /api/productos. Deberías recibir la misma colección de tres elementos mientras el proceso API no se haya reiniciado. Esto demuestra que los productos no están guardados en el componente ActualizarRemoto.

Ahora crea otro producto utilizando una solicitud HTTP independiente. La página Blazor seguirá mostrando la lista anterior hasta que el alumno consulte de nuevo. No es necesario cambiar código para repetir el experimento con distintos números de elementos.

Paso R8. Reiniciar el proceso servidor

Detén completamente la API, sin detener Blazor. Vuelve a iniciarla. El diccionario temporal del servidor se reconstruirá con Cuaderno y Lápiz.

La pantalla /actualizar-remoto puede seguir mostrando tres o cuatro productos porque esa colección está en el estado del componente. Pulsa Consultar servidor: la lista debe regresar a dos productos. El comportamiento enseña que **la copia visual y la memoria del servidor tienen ciclos de vida distintos**.

No reescribas el componente para borrar automáticamente productos al detectar cualquier error. Si GET falla, la interfaz puede decidir conservar el último dato conocido y mostrar un aviso. Nuestra versión no borra la lista en el catch, precisamente para distinguir «no se obtuvo una actualización» de «la API devolvió una lista vacía».

Paso R9. Distinguir errores sin borrar datos válidos anteriores

Con la página mostrando dos productos, detén la API y pulsa Consultar. El componente intentará GET, recibirá un error de conexión y mostrará un mensaje. La lista anterior puede permanecer visible porque representa la última consulta que sí terminó correctamente.

Explica por qué sería engañoso mostrar «Catálogo vacío» en ese instante: no hemos recibido una respuesta que confirme cero productos. Hemos perdido la capacidad de consultar. Si la pantalla decide conservar la última lista, debería informar además de que esos datos podrían estar desactualizados.

Después inicia de nuevo la API y pulsa Consultar. El error debe desaparecer y la lista debe reflejar la respuesta nueva. No borres manualmente la variable error antes de cada petición sin comprobar qué sucederá si la siguiente consulta falla: el ejercicio la limpia al comienzo de una operación nueva y la vuelve a establecer si ocurre un problema.

Paso R10. Convertir la lectura en un experimento de diseño

La página usa IHttpClientFactory directamente, igual que el primer estado de 4.1. Pregúntate qué ocurriría si quisieras añadir el mismo botón a tres páginas distintas. Cada una tendría que conocer nombre del cliente, ruta y deserialización.

Esta repetición es precisamente el motivo del refactor de 4.2. En el estado posterior, la página podría inyectar IProductosApiService y ejecutar ObtenerAsync en lugar de utilizar GetFromJsonAsync. El comportamiento visual seguiría siendo el mismo y los detalles de HTTP quedarían concentrados en una clase.

No es obligatorio hacer ese segundo refactor sobre ActualizarRemoto.razor: el alumno ya lo realizará con CatalogoRemoto. El propósito de esta variante consiste en que la necesidad de separar responsabilidades surja de una situación observada.

Paso R11. Reconocer los límites de esta solución

Consultar manualmente no es una estrategia universal para una aplicación empresarial. En algunos sistemas se necesitan notificaciones del servidor, actualizaciones automáticas, gestión de caché o detección de cambios concurrentes. En nuestro ejercicio no aparecen esos requisitos y no debemos añadir infraestructura que no está justificada.

La solución enseña una idea transferible: un componente consume una representación remota en un instante; si desea datos posteriores, tiene que establecer una estrategia para obtenerlos. El uso de HTTP no convierte por sí mismo una lista en un estado sincronizado en tiempo real.

Matriz de comprobación de la ampliación de actualización

Acción	Estado real de API	Estado visible antes de consultar	Después de consultar
Arranque limpio	Dos productos	Ninguna consulta	Dos productos
POST Regla desde PowerShell	Tres productos	Dos productos	Tres productos
Segundo POST externo	Cuatro productos	Tres productos	Cuatro productos
Reinicio de API	Dos productos	Última lista visible	Dos productos
API apagada	No responde	Última lista visible	Mensaje de error

Esta actividad complementaria debe concluir con el alumno capaz de explicar la diferencia entre el estado del servidor, el estado de la interfaz, una respuesta HTTP confirmada y una petición que no pudo completarse. No es necesario añadir un framework de sincronización para demostrarlo.

Ampliación autónoma P4.1 — Comparar el cliente de servidor con fetch del navegador

En los pasos obligatorios, Blazor Interactive Server realiza peticiones desde su proceso ASP.NET Core mediante HttpClient. Ahora ejecutaremos una consulta similar desde JavaScript del navegador para identificar una diferencia que suele confundirse: **CORS depende del origen de la petición y del cliente que la realiza**.

Este taller no es necesario para construir el catálogo remoto. No habilites CORS en la API del proyecto de referencia por defecto. Trabajaremos con una copia de la API y utilizaremos temporalmente la consola JavaScript del navegador para observar el comportamiento.

Paso C1. Arrancar la API y Blazor en puertos distintos

Inicia la API en http://localhost:5081 y comprueba con curl.exe que GET /api/productos responde 200 con los productos iniciales. Después inicia Blazor en http://localhost:5091 y abre /catalogo-remoto.

La página debe poder consultar los productos desde su componente C# mientras el servicio con nombre y la URL estén correctamente configurados. Este es el funcionamiento de referencia: Blazor utiliza HttpClient desde el host servidor.

Anota mentalmente los dos puertos. Aunque ambos utilicen localhost, el origen de la página en 5091 no es igual al de la API en 5081.

Paso C2. Abrir herramientas de desarrollo del navegador

Desde la página de Blazor abre las herramientas de desarrollo de tu navegador y selecciona la consola JavaScript. No cierres la página ni modifiques sus componentes. La consola ejecuta código en el navegador, no dentro del proceso .NET del servidor Blazor.

Escribe:

```
fetch("http://localhost:5081/api/productos")
  .then(r => r.json())
  .then(datos => console.log(datos))
  .catch(error => console.error("Consulta del navegador:", error));
```

La solicitud se origina ahora en el navegador desde una página cuyo origen incluye el puerto 5091. Si la API no publica una política CORS que permita a ese origen leer la respuesta, el navegador puede informar de una restricción de origen cruzado.

No concluyas que el endpoint GET ha dejado de funcionar por el mero mensaje de la consola. La petición directa desde curl.exe puede continuar devolviendo 200.

Paso C3. Repetir la prueba con un cliente independiente del navegador

Desde PowerShell ejecuta:

```
curl.exe -i http://localhost:5081/api/productos
```

Deberías seguir observando el catálogo. Este comando no ejecuta la política CORS propia de una página JavaScript. Su resultado permite separar el funcionamiento del servidor de las restricciones de acceso que el navegador aplica al código de otro origen.

Vuelve a /catalogo-remoto y consulta desde el componente Blazor. El cliente HttpClient de Interactive Server también debe seguir funcionando. La API puede estar operativa para ambos consumidores y, al mismo tiempo, impedir que un script de otro origen lea el resultado.

Paso C4. Crear una copia para una política CORS restringida

Conserva la API original. En una copia de la API de 4.3, antes de builder.Build agrega:

```
builder.Services.AddCors(opciones =>
{
    opciones.AddPolicy("ClienteDesarrollo", politica =>
        politica.WithOrigins("http://localhost:5091")
                .AllowAnyHeader()
                .WithMethods("GET", "POST"));
});
```

Después de construir app y antes de ejecutar los endpoints, aplica:

```
app.UseCors("ClienteDesarrollo");
```

Compila y arranca esta copia en 5081, deteniendo previamente el proceso API que utilizaba el puerto. La política representa una elección específica de origen y verbos, no un permiso global para cualquier cliente web.

Paso C5. Repetir fetch desde el origen permitido

Abre de nuevo la página alojada en 5091 y ejecuta la consulta JavaScript anterior. Si la política y los orígenes coinciden, el navegador debe poder acceder al JSON de la API. Comprueba que aparecen Cuaderno y Lápiz.

Ahora cambia la URL de Blazor para abrir la misma aplicación desde otro host o puerto, si tienes preparada esa variante. Ese origen distinto no está incluido automáticamente en la política. El resultado sirve para explicar que permitir un origen concreto no equivale a permitir cualquiera.

No modifiques la política para aceptar todos los orígenes solo con el fin de eliminar un mensaje de error. La finalidad de este paso es observar que la configuración responde a un requisito explícito del cliente.

Paso C6. Diferenciar CORS de autorización

La API de productos sigue siendo pública en el laboratorio. Si una página JavaScript tiene permiso CORS para leer su respuesta, eso no significa que se haya autenticado como Formador ni que el servidor conozca una identidad de usuario.

Del mismo modo, una API protegida mediante una política de autorización no debería conceder acceso a una identidad sin permisos porque su origen estuviera permitido por CORS. Son mecanismos distintos y pertenecen a preguntas diferentes.

El componente AuthorizeView del punto 4.4 tampoco cambia la política CORS del servidor API. Una regla visual aplicada en Blazor no configura cómo el navegador debe tratar respuestas HTTP de otro origen.

Paso C7. Reconocer una solicitud preliminar

Desde el navegador podrías intentar una operación POST con Content-Type application/json. Dependiendo de las condiciones de la solicitud, el navegador puede emitir antes una petición OPTIONS de comprobación. Esa solicitud no crea productos; pregunta si el origen tiene permiso para realizar la operación propuesta.

En las herramientas de desarrollo del navegador examina las solicitudes de red y distingue OPTIONS de POST. No supongas que la presencia de una solicitud preliminar demuestra que el endpoint de creación recibió ya el cuerpo JSON.

En esta ampliación no es necesario crear productos desde fetch; basta reconocer que la secuencia de red puede ser distinta de la utilizada por HttpClient entre procesos de servidor.

Paso C8. Restaurar el estado del curso

Cuando termines, detén la copia con CORS opcional y vuelve a utilizar la API original. El componente Blazor debe continuar consumiendo GET y POST sin necesitar que la API acepte expresamente el origen de su página.

La conclusión formativa consiste en saber identificar dónde nació la petición. Si procede de C# en Interactive Server, analizamos comunicación HTTP entre servidores. Si procede de fetch en el navegador, puede ser necesario examinar CORS. Esta diferencia evita cambios de configuración innecesarios y diagnósticos incorrectos.

Comparación final de los clientes

Cliente	Dónde se ejecuta	Política CORS de navegador
curl.exe	Terminal local	No la aplica
HttpClient de Blazor Interactive Server	Servidor Blazor	No la aplica
fetch en consola JavaScript	Navegador	Puede aplicarla

Esta matriz no convierte CORS en una alternativa a autenticación ni autorización. Sirve para comprender que un mismo endpoint puede producir observaciones distintas según el contexto en que se ejecuta el cliente.

Resultado y continuidad

Al finalizar, `/productos` conserva el aprendizaje local de M03 y `/catalogo-remoto` demuestra GET, POST y errores HTTP entre dos procesos. La API auxiliar utilizada aquí se estudiará en detalle en P4.3; en **P4.2** extraeremos las llamadas del componente a `IProductosApiService`.

P4.2 — Extraer las llamadas HTTP a un servicio inyectable

Punto de partida

Trabaja sobre una copia del estado `PROYECTO/4.1/CursoWeb.Blazor`. No sobrescribas el proyecto de 4.1 ni `M03_BLAZOR`. Conserva en ejecución la **misma API auxiliar** `PROYECTO/4.1/CursoWeb.ProductosApi` en `http://localhost:5081`. En `/catalogo-remoto` deben verse dos productos antes de empezar.

Resultado esperado: el componente deja de conocer `HttpClient` y las rutas HTTP; un servicio tipado ejecuta la comunicación. El comportamiento visible de la página debe seguir siendo el mismo.

Paso 4.2.1. Declarar una interfaz explícita

Crea `Services/IProductosApiService.cs`:

```
using CursoWeb.Blazor.Models;

namespace CursoWeb.Blazor.Services;

public interface IProductosApiService
{
    Task<IReadOnlyList<ProductoDto>> ObtenerAsync();
    Task<ProductoDto> CrearAsync(CrearProductoDto solicitud);
}
```

Separa en tu explicación oral qué recibe la pantalla (`IReadOnlyList<ProductoDto>`) de cómo se obtuvo (desconocido para la interfaz). Compara esta interfaz con `IProductosMemoriaService` de M03; **no borres** ese servicio antiguo porque sigue siendo parte del aprendizaje previo.

Paso 4.2.2. Implementar la comunicación en una clase

Crea `Services/ProductosApiService.cs` con el código íntegro que aparece a continuación en esta práctica. Debe implementar **ambos métodos**. Al terminar, comprueba que en esa clase aparecen `GetFromJsonAsync`, `PostAsJsonAsync` y `EnsureSuccessStatusCode`.

No envíes desde el cliente un ID inventado en `CrearProductoDto`: la API asignará el valor y devolverá `ProductoDto`.

Código íntegro del servicio HTTP (paso 4.2.2)

Crea este archivo en `Services/ProductosApiService.cs`, además de la interfaz del paso 4.2.1. Este es el código que compilará al resolver la dependencia `IProductosApiService` mediante DI:

```
using System.Net.Http.Json;
using CursoWeb.Blazor.Models;

namespace CursoWeb.Blazor.Services;

public sealed class ProductosApiService(HttpClient cliente) : IProductosApiService
{
```

```
    public async Task<IReadOnlyList<ProductoDto>> ObtenerAsync()
    {
        return await cliente.GetFromJsonAsync<List<ProductoDto>>(
            "api/productos") ?? [];
    }

    public async Task<ProductoDto> CrearAsync(CrearProductoDto solicitud)
    {
        using var respuesta = await cliente.PostAsJsonAsync(
            "api/productos", solicitud);
        respuesta.EnsureSuccessStatusCode();

        return await respuesta.Content.ReadFromJsonAsync<ProductoDto>()
               ?? throw new InvalidOperationException(
                   "La API confirmó un alta sin devolver un producto.");
    }
}
```

Comprobación intermedia: antes de modificar la página, registra el servicio como se muestra en el paso siguiente y compila. La clase depende de que existan `IProductosApiService`, `ProductoDto` y `CrearProductoDto`. Si falta un `using` o el namespace no coincide, el compilador lo indicará antes de que intervenga HTTP.

Paso 4.2.3. Registrar el typed client

En `Program.cs` elimina el registro de cliente con nombre `AddHttpClient("ProductosApi", ...)` que se utilizó en 4.1 y coloca el registro tipado `AddHttpClient<IProductosApiService, ProductosApiService>(...)` de teoría 4.2.3.

Mantén los registros de componentes de Blazor y `IProductosMemoriaService`. Conserva la clave de configuración `ProductosApi:BaseUrl` y el puerto 5081. Si quitas accidentalmente el servicio de memoria, puedes provocar una regresión en la página `/productos`.

Paso 4.2.4. Sustituir el acceso HTTP de la página

En `Components/Pages/CatalogoRemoto.razor` cambia:

```
@inject IHttpClientFactory Clientes
```

por:

```
@inject IProductosApiService CatalogoApi
```

En `OnInitializedAsync`, sustituye la creación del cliente y la llamada `GetFromJsonAsync` por:

```
productos = (await CatalogoApi.ObtenerAsync()).ToList();
```

En `CrearAsync`, sustituye la construcción de `HttpClient`, `PostAsJsonAsync` y `ReadFromJsonAsync` por:

```
var creado = await CatalogoApi.CrearAsync(
    new CrearProductoDto(nombreNuevo.Trim(), precioNuevo));
```

No cambies el marcado HTML, la validación preliminar ni los mensajes de error. La finalidad de este punto es aprender a cambiar responsabilidades internas **sin cambiar la experiencia del usuario**.

Paso 4.2.5. Comparar el comportamiento

Ejecuta la API y Blazor por separado. Desde el navegador abre `/catalogo-remoto`, comprueba los dos productos y crea «Regla» con precio 1,75. Refresca la página y verifica que la nueva entrada sigue en memoria de la API. Después detén la API y recarga: la pantalla debe mostrar el aviso de fallo, en lugar de afirmar que el catálogo está vacío. Reinicia la API al acabar.

Abre también `/productos` y comprueba que continúa siendo accesible. Si algo falla, investiga primero el registro de servicios, el puerto de la API y la clave `ProductosApi:BaseUrl`.

Paso 4.2.6. Reto: comprobar que el componente ignora la ruta

Localiza todos los lugares de `CatalogoRemoto.razor` donde aparezca la cadena `"api/productos"`. **El resultado debe ser ninguno**. Localiza ahora esa ruta en `ProductosApiService.cs`: aquí sí debe aparecer, porque es un detalle de comunicación.

Como variante de exploración, cambia solo esa ruta por `"api/no-existe"`, ejecuta de nuevo y observa el aviso de error HTTP. Restablece después `"api/productos"`.

Taller de refactor P4.2: comprobar las fronteras antes y después

El objetivo de la práctica no es añadir una segunda pantalla; es transformar el diseño conservando resultados. Antes de tocar el código, escribe qué se mantiene invariable: al abrir el catálogo remoto aparecen los productos, al crear Regla aparece una confirmación, el servidor puede rechazar datos inválidos y la falta de conexión se presenta como error. El catálogo local del módulo 3 sigue funcionando por separado.

Inventario anterior. En el archivo de P4.1 el componente conoce IHttpClientFactory, el nombre ProductosApi, la ruta api/productos y los métodos GetFromJsonAsync y PostAsJsonAsync. Estas dependencias funcionan, pero convierten a la presentación en responsable de detalles de comunicación. El ejercicio consiste en desplazar esos detalles a una clase sin alterar las decisiones visuales.

Primera transformación. Crea la interfaz IProductosApiService con dos operaciones. No agregues todavía un tercer método que no se utilice. La firma de ObtenerAsync devuelve una colección de solo lectura; la firma de CrearAsync recibe un objeto sin ID y devuelve un objeto completo con ID. Comprueba que el proyecto compila después de crear la interfaz y antes de cambiar el componente.

Segunda transformación. Implementa la clase ProductosApiService. La clase recibe HttpClient por su constructor primario y usa exactamente las rutas del contrato. Si el proyecto no compila, revisa los espacios de nombres y las declaraciones de los DTO antes de añadir manejo de errores o cambiar la URL. De esta forma separas problemas de tipos de problemas de HTTP.

Tercera transformación. Sustituye el registro con nombre por el registro tipado. No dejes ambos registros funcionando para el mismo ejemplo: si una variante vieja continúa presente, el alumno podría creer que la página está usando una configuración que en realidad ya no se consulta. Conserva en cambio los servicios de Blazor y el servicio de memoria de M03.

Cuarta transformación. En CatalogoRemoto.razor cambia la inyección de IHttpClientFactory por IProductosApiService, reemplaza el GET por ObtenerAsync y sustituye el bloque HTTP del POST por CrearAsync. Debe seguir existiendo una validación de nombre y precio en el componente y una notificación visual tras el alta. El servicio nuevo no se encarga de dibujar mensajes.

Quinta transformación. Arranca ambos procesos y compara el comportamiento con P4.1. Si funciona igual, la refactorización ha conservado los resultados. Si ha cambiado, investiga primero la transformación concreta que acabas de ejecutar en vez de modificar a la vez el servidor.

Program.cs completo del estado P4.2

El siguiente archivo muestra el registro del cliente tipado en el contexto real de la aplicación. Conserva los servicios utilizados por las páginas de M03. Léelo después de completar el registro manual para contrastar qué líneas han cambiado.

```
using CursoWeb.Blazor.Components;
using CursoWeb.Blazor.Services;

var builder = WebApplication.CreateBuilder(args);
// Servicios necesarios para componentes Razor y circuitos Interactive Server.
builder.Services.AddRazorComponents().AddInteractiveServerComponents();
// PASO 3.6.1: inyección por interfaz. No existe HttpClient ni API aún.
builder.Services.AddScoped<IProductosMemoriaService, ProductosMemoriaService>();
// P4.1: cliente HTTP con nombre, consumido desde la pantalla remota.
// P4.2: cliente tipado; el componente solo conoce IProductosApiService.
builder.Services.AddHttpClient<IProductosApiService, ProductosApiService>(cliente =>
{
    cliente.BaseAddress = new Uri(
        builder.Configuration["ProductosApi:BaseUrl"] ?? "http://localhost:5081/");
    cliente.Timeout = TimeSpan.FromSeconds(10);
});
var app = builder.Build();
if (!app.Environment.IsDevelopment()) app.UseExceptionHandler("/error");
app.UseStaticFiles();
app.UseAntiforgery();
// El modo Interactive Server necesita tanto servicios como endpoint.
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
app.Run();
public partial class Program { }
```

CatalogoRemoto.razor completo después del refactor

Comprueba que la página ya no crea ni nombra un HttpClient. El componente conoce ObtenerAsync y CrearAsync, mantiene cargando, enviando y los mensajes de estado, y conserva sus eventos Razor. Todo acceso a la ruta HTTP se concentra en ProductosApiService.

```
@page "/catalogo-remoto"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject IProductosApiService CatalogoApi
<PageTitle>Productos desde API</PageTitle>

<h1>Catálogo remoto · P4.1</h1>
<p>Esta página consume una API HTTP. El catálogo local de M03 sigue disponible en /productos.</
p>
@if (cargando)
{
    <p id="estado-api" role="status">Cargando catálogo remoto...</p>
}
```

```
else if (error is not null)
{
    <p id="error-api" role="alert">@error</p>
}
else
{
    <p id="estado-api">Catálogo remoto disponible</p>
    <ul id="productos-api">
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
        }
    </ul>
}
<section>
    <h2>Crear producto</h2>
    <label>Nombre: <input id="nombre-api" @bind="nombreNuevo" /></label>
    <label>Precio: <input id="precio-api" type="number" min="0" step="0.01"
@bind="precioNuevo" /></label>
    <button id="crear-api" disabled="@enviando" @onclick="CrearAsync">Crear en API</button>
    @if (mensaje is not null)
    {
        <p id="resultado-api" role="status">@mensaje</p>
    }
</section>

@code {
    private List<ProductoDto> productos = [];
    private bool cargando = true;
    private string? error;
    private string nombreNuevo = string.Empty;
    private decimal precioNuevo;
    private bool enviando;
    private string? mensaje;

    protected override async Task OnInitializedAsync()
    {
        try
        {
            productos = (await CatalogoApi.ObtenerAsync()).ToList();
        }
        catch (HttpRequestException)
        {
            error = "No se pudo obtener el catálogo remoto.";
        }
        catch (TaskCanceledException)
        {
            error = "La petición superó el tiempo disponible.";
        }
        catch (System.Text.Json.JsonException)
        {
            error = "La API no devolvió un catálogo JSON válido.";
        }
        finally
        {
            cargando = false;
        }
    }

    private async Task CrearAsync()
    {
        if (string.IsNullOrWhiteSpace(nombreNuevo) || precioNuevo <= 0)
        {
            mensaje = "Indica un nombre y un precio mayor que cero.";
            return;
```

```
        }

        enviando = true;
        mensaje = null;
        try
        {
            var creado = await CatalogoApi.CrearAsync(new CrearProductoDto(nombreNuevo.Trim(),
precioNuevo));
            productos.Add(creado);
            nombreNuevo = string.Empty;
            precioNuevo = 0;
            mensaje = $"Producto creado: {creado.Nombre} (id {creado.Id}).";
        }
        catch (HttpRequestException)
        {
            mensaje = "La API no pudo completar el alta.";
        }
        catch (TaskCanceledException)
        {
            mensaje = "Se agotó el tiempo al crear el producto.";
        }
        catch (System.Text.Json.JsonException)
        {
            mensaje = "La respuesta de alta no contiene JSON válido.";
        }
        catch (InvalidOperationException)
        {
            mensaje = "La respuesta de alta está incompleta.";
        }
        finally
        {
            enviando = false;
        }
    }

    /* RETO P4.2: comprobar que la ruta "api/productos" solo aparece en
       ProductosApiService, nunca en las llamadas del componente. */
}
```

P4.2 — Ejercicios de regresión guiada

Escenario 1: los datos siguen siendo remotos

Arranca la API en el puerto 5081 y abre /catalogo-remoto. Debes ver dos productos. Detén la API y recarga: debe aparecer una condición de fallo. Si continúan apareciendo los dos productos sin ninguna petición, comprueba si estás observando por error la página local /productos o una copia antigua de la aplicación.

Escenario 2: la creación conserva su contrato

Reinicia la API y crea Regla por 1,75. El mensaje debe incluir el nombre y el ID real. Recarga la página y verifica que la entrada continúa en la colección de memoria del servidor. Después reinicia la API y comprueba que el catálogo vuelve a dos elementos. Esa persistencia temporal pertenece al proceso API, no al servicio tipado ni al componente.

Escenario 3: la interfaz ya no contiene rutas HTTP

Busca la cadena api/productos en CatalogoRemoto.razor. El resultado esperado es cero. Haz la búsqueda en ProductosApiService.cs; el resultado debe localizar los métodos correspondientes a

consulta y creación. Ahora cambia temporalmente la ruta en el servicio, ejecuta y observa el fallo. Restablece después el contrato original.

No elimines el estado error del componente por considerar que el servicio se ocupa de las excepciones. La clase de infraestructura puede identificar un problema de HTTP, pero la vista sigue siendo responsable de informar al usuario.

Escenario 4: una dependencia sin registrar

Como ejercicio razonado, imagina que comentamos por accidente AddHttpClient<IProductosApiService, ProductosApiService> pero el componente continúa solicitando IProductosApiService. Antes de probar, predice el resultado: el problema aparecerá al resolver la dependencia, antes de que se envíe una petición HTTP. Restaurar la configuración de DI soluciona la causa; cambiar el puerto no tendría efecto.

Escenario 5: el servidor funciona pero el puerto no coincide

Mantén el registro de DI correcto y cambia temporalmente la URL base para apuntar a un puerto en el que no hay API. El componente podrá obtener una instancia de su servicio, pero la consulta no llegará al servidor esperado. Este resultado es diferente del escenario anterior y permite distinguir configuración de dependencias y conectividad.

Escenario 6: compatibilidad de DTO

Examina el JSON de respuesta de GET y compara sus campos con ProductoDto. Reflexiona sobre qué ocurriría si la API enviara un objeto de resumen en vez de una lista. La implementación de ProductosApiService es el lugar donde se detectaría el desacuerdo de contrato, mientras que la vista transformaría la incidencia en un mensaje visible.

Escenario 7: regresión del módulo 3

Abre la página local /productos. Debe continuar operativa, porque mantenemos IProductosMemoriaService y los componentes originales. Si esa pantalla falla al iniciar, revisa los servicios que has eliminado en Program.cs: la práctica no autorizaba retirar ninguna dependencia utilizada por el estado anterior.

Matriz de responsabilidades tras P4.2

Responsabilidad	Componente Razor	Servicio tipado	API
Dibujar lista	Sí	No	No
Leer valores del formulario	Sí	No	No
Elegir endpoint HTTP	No	Sí	Publica ruta
Configurar HttpClient	No	Registro DI	No
Serializar solicitud JSON	No	Sí	Deserializa
Asignar identificador	No	No	Sí
Validar entrada del servidor	No sustituye servidor	No	Sí
Mostrar error al usuario	Sí	Comunica fallo	Devuelve estado

La evidencia final no es simplemente que un archivo compiló. Es que el comportamiento visible se mantiene y que el componente dejó de conocer detalles HTTP. Al cumplir ambas condiciones hemos efectuado un refactor que el alumno puede justificar.

Ampliación P4.2 — Taller adicional: demostrar la DI cambiando una implementación

Hasta ahora sabemos que CatalogoRemoto.razor no debería conocer detalles HTTP después del refactor. Vamos a comprobar esa propiedad con un experimento distinto de los anteriores: **sustituir el origen de los productos sin tocar el componente**. La finalidad no es crear otra aplicación de negocios, sino poner a prueba el significado de una interfaz inyectada.

Trabaja en una copia del proyecto 4.2, conservando intacto el estado que consume la API real. En la carpeta Services crea ProductosApiSimuladoService.cs. La clase implementará los mismos dos métodos de IProductosApiService, pero no tendrá HttpClient, rutas ni serialización JSON. Inicialmente ofrecerá dos objetos conocidos: Libreta simulada y Bolígrafo simulado.

Esta implementación debe utilizar los mismos DTO públicos que espera la página, incluido un identificador asignado al crear otro elemento. De ese modo la UI no distingue los datos por su tipo C#, sino por el origen de la implementación que resuelve la interfaz.

Paso S1. Escribir la clase que satisface el contrato

El código siguiente también está disponible como archivo docente en el proyecto 4.2, pero debes introducirlo en la copia de trabajo para comprobar cómo la interfaz obliga a proporcionar las firmas correctas.

```
using CursoWeb.Blazor.Models;

namespace CursoWeb.Blazor.Services;

// RETO DOCENTE P4.2: implementación alternativa sin comunicación HTTP.
// Se mantiene sin registrar por defecto; la app sigue usando ProductosApiService.
// Para activarla se sustituye temporalmente el registro de IProductosApiService
// en Program.cs; NO se registran las dos implementaciones a la vez.
public sealed class ProductosApiSimuladoService : IProductosApiService
{
    private readonly List<ProductoDto> productos =
    [
        new ProductoDto(101, "Libreta simulada", 4.25m),
        new ProductoDto(102, "Bolígrafo simulado", 1.10m)
    ];

    private int siguienteId = 102;

    public Task<IReadOnlyList<ProductoDto>> ObtenerAsync()
    {
        IReadOnlyList<ProductoDto> resultado = productos.ToArray();
        return Task.FromResult(resultado);
    }

    public Task<ProductoDto> CrearAsync(CrearProductoDto solicitud)
    {
        if (string.IsNullOrWhiteSpace(solicitud.Nombre) || solicitud.Precio <= 0)
            throw new ArgumentException("Nombre y precio deben ser válidos.");

        var creado = new ProductoDto(
            Interlocked.Increment(ref siguienteId),
            solicitud.Nombre.Trim(),
```

```
            solicitud.Precio);
        productos.Add(creado);
        return Task.FromResult(creado);
    }
}
```

No confundas el nombre ProductosApiSimuladoService con una API adicional. Esta clase no publica HTTP; vive dentro del host Blazor, igual que el componente. Su finalidad es responder a la misma abstracción sin consumir una red.

Paso S2. Cambiar solamente el registro

En Program.cs localiza el registro del cliente tipado. Coméntalo temporalmente y registra en su lugar:

```
builder.Services.AddScoped<IProductosApiService, ProductosApiSimuladoService>();
```

Mantén AddRazorComponents, AddInteractiveServerComponents y el servicio IProductosMemoriaService del módulo 3. No sustituyas todos los registros por una implementación distinta; solo cambia la asignación de IProductosApiService.

La elección AddScoped permite que la instancia de demostración mantenga una colección propia mientras dura el ámbito correspondiente. No es un almacenamiento persistente. En Blazor Interactive Server la duración del ámbito está ligada al circuito y no debe equipararse a una base de datos compartida por todos los navegadores.

Paso S3. Consultar sin arrancar la API

Detén el proceso de la API auxiliar y arranca únicamente Blazor desde esta copia. Abre /catalogo-remoto. Ahora deben aparecer Libreta simulada y Bolígrafo simulado, no Cuaderno y Lápiz. La pantalla sigue mostrando su lista y su formulario, pero el origen es otro.

Este resultado es una demostración de desacoplamiento. El componente no ha cambiado la ruta, no ha introducido un condicional especial ni ha tenido que conocer la clase concreta. El contenedor ha proporcionado una implementación diferente de la misma interfaz.

Paso S4. Crear un producto con la implementación simulada

Escribe Agenda simulada y precio 6,40. El botón sigue ejecutando CrearAsync desde el componente, pero la implementación del contrato trabaja sobre su lista local. Debe generar un identificador de demostración y devolver un ProductoDto con el nombre y el precio normalizados.

La página debe añadir el objeto recibido y presentar el mensaje de confirmación. Observa qué no ha ocurrido: no existe petición a /api/productos ni respuesta 201. Este experimento valida el acoplamiento del componente a una interfaz, **no** sustituye las pruebas HTTP realizadas con el servidor real.

Paso S5. Restaurar la comunicación remota

Retira el registro simulado y restaura el registro tipado:

```
builder.Services.AddHttpClient<IProductosApiService, ProductosApiService>(
    cliente =>
    {
        cliente.BaseAddress = new Uri(
```

```
            builder.Configuration["ProductosApi:BaseUrl"]
            ?? "http://localhost:5081/");
        cliente.Timeout = TimeSpan.FromSeconds(10);
    });
```

Arranca de nuevo la API y después Blazor. El catálogo debe volver a mostrar Cuaderno y Lápiz. Verifica que la clase simulada sigue existiendo como variante pedagógica pero no está registrada por defecto.

Paso S6. Provocar un defecto de DI que no sea HTTP

Comenta temporalmente ambos registros de IProductosApiService y ejecuta la aplicación. Al cargar un componente que solicita esa interfaz, el host no puede resolver su dependencia. Eso es distinto de recibir 404 o una excepción por conexión; ni siquiera hemos llegado a iniciar una solicitud HTTP.

Restablece el registro real y comprueba que la página vuelve a funcionar. Este contraste permite clasificar incidencias: clase no registrada, cliente configurado con puerto incorrecto, servidor que devuelve un rechazo y cuerpo JSON incompatible.

Paso S7. Explicar por qué el comportamiento no demuestra persistencia

Crea un producto con el servicio simulado y abre la página desde otra sesión o reinicia Blazor. No presupongas que aparecerá la misma colección. El servicio simulado utiliza una lista en memoria dentro de la instancia que ha proporcionado DI. No comparte automáticamente una base de datos y no se ha diseñado como almacenamiento global.

El objetivo de este paso es comprender que sustituir el servicio mantiene la interfaz, pero puede modificar la procedencia, duración y consistencia de los datos. La abstracción define operaciones y tipos; no convierte por sí sola todas las implementaciones en equivalentes bajo cualquier condición de persistencia.

P4.2 — Interpretar las cuatro pruebas de desacoplamiento

Ensayo	Fuente que resuelve IProductosApiService	Necesita API HTTP	Resultado visible
Estado habitual	ProductosApiService	Sí	Cuaderno y Lápiz
Variante simulada	ProductosApiSimuladoService	No	Libreta y Bolígrafo simulados
Sin registrar interfaz	Ninguna	No llega a usarla	Error de resolución de dependencia
Registro real con puerto incorrecto	ProductosApiService	Intenta usarla	Error de comunicación

Tras completar la tabla, vuelve a la implementación real. Debes poder explicar que el objetivo de P4.2 no era eliminar la comunicación HTTP, sino ponerla en una clase dedicada que puede intercambiarse sin reescribir la página.

P4.2 — Reconstruir mentalmente la cadena de llamadas

Cuando el usuario abre /catalogo-remoto, Blazor activa el componente que solicita IProductosApiService. El contenedor consulta su configuración, crea o resuelve ProductosApiService

y proporciona un HttpClient con la dirección base configurada. El componente invoca ObtenerAsync, el servicio realiza GET, la API devuelve el JSON y el componente actualiza la colección.

Cuando sustituimos el registro, las dos primeras decisiones son diferentes: el contenedor entrega ProductosApiSimuladoService, que construye la lista local y la devuelve de inmediato. El componente sigue llamando al mismo método. Por eso conserva el código de representación de tarjetas, mensajes y botones.

Esa narración deja claro que la inyección de dependencias no es una tecnología aislada del comportamiento. Se utiliza para definir quién decide qué implementación proporciona los productos. Esa decisión permite refactorizar, enseñar y probar separadamente componentes y comunicación.

Ampliación autónoma P4.2 — Probar un cliente HTTP sin arrancar una API

El servicio simulado que hemos estudiado en P4.2 sustituye por completo ProductosApiService. Es útil para comprobar que el componente se desacopla de la comunicación, pero no demuestra que la implementación HTTP concreta utilice el verbo y la ruta correctos. En esta ampliación crearemos un **cliente de pruebas muy pequeño con un HttpMessageHandler controlado** para comprobar esa segunda responsabilidad.

No utilizaremos un framework de mocking ni instalaremos librerías adicionales. Trabajaremos con C# y HttpClient, que el alumnado ya conoce. El objetivo no es automatizar toda una aplicación, sino observar qué parte del comportamiento puede probarse sin disponer de un servidor escuchando.

Paso H1. Crear una aplicación de consola de ensayo

Desde PowerShell crea un proyecto independiente:

```
dotnet new console -n ClienteHttpLaboratorio -f net8.0
cd ClienteHttpLaboratorio
```

No copies todo el proyecto Blazor a esta nueva carpeta. El laboratorio solo necesita un cliente HTTP, los DTO mínimos y un manejador que devuelva respuestas simuladas. Evitar dependencias innecesarias hace que el alumno pueda aislar el concepto.

El programa no debe modificar las rutas del proyecto ASP.NET Core ni ocupar el puerto 5081. Toda la respuesta se producirá dentro del propio proceso de consola, por lo que puede ejecutarse aun cuando la API esté detenida.

Paso H2. Comprender el contrato que queremos comprobar

Queremos que una operación ObtenerAsync envíe GET a /api/productos y convierta un JSON de lista en una colección de DTO. El resultado conocido de la prueba será un objeto Cuaderno con ID 1 y precio 2,50.

La API real no interviene. Lo que vamos a comprobar es que el código de cliente construye la solicitud prevista y sabe interpretar una respuesta con el cuerpo que hemos acordado.

Para realizar esta observación, HttpClient puede recibir un HttpMessageHandler. Nuestro manejador registrará el método y la ruta y devolverá un HttpResponseMessage definido por el propio ejercicio.

Paso H3. Construir Program.cs íntegro

Sustituye el contenido de Program.cs por el siguiente programa, que contiene los tipos mínimos necesarios. Es autocontenido y no requiere paquetes externos:

```
using System.Net;
using System.Net.Http.Json;
using System.Text;

using var manejador = new ManejadorDeLaboratorio();
using var cliente = new HttpClient(manejador)
{
    BaseAddress = new Uri("http://laboratorio.local/")
};

var productos = await ObtenerAsync(cliente);
if (productos.Count != 1 || productos[0].Nombre != "Cuaderno")
    throw new InvalidOperationException("El JSON no coincide con el DTO.");

if (manejador.UltimoMetodo != HttpMethod.Get ||
    manejador.UltimaRuta != "/api/productos")
    throw new InvalidOperationException("La solicitud no usa el contrato.");

Console.WriteLine("GET y deserialización correctos.");

static async Task<List<ProductoDto>> ObtenerAsync(HttpClient cliente)
{
    return await cliente.GetFromJsonAsync<List<ProductoDto>>(
        "api/productos") ?? [];
}

record ProductoDto(int Id, string Nombre, decimal Precio);

sealed class ManejadorDeLaboratorio : HttpMessageHandler
{
    public HttpMethod? UltimoMetodo { get; private set; }
    public string? UltimaRuta { get; private set; }

    protected override Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage solicitud,
        CancellationToken cancelacion)
    {
        UltimoMetodo = solicitud.Method;
        UltimaRuta = solicitud.RequestUri?.AbsolutePath;

        var respuesta = new HttpResponseMessage(HttpStatusCode.OK)
        {
            Content = new StringContent(
                "[{\"id\":1,\"nombre\":\"Cuaderno\",\"precio\":2.50}]",
                Encoding.UTF8,
                "application/json")
        };
        return Task.FromResult(respuesta);
    }
}
```

El método ObtenerAsync utiliza la misma operación HTTP que la primera versión del componente, pero no presenta HTML. La clase ManejadorDeLaboratorio hereda de HttpMessageHandler y decide qué HttpResponseMessage devolver. El programa comprueba los datos deserializados y la forma de la solicitud.

Si el alumno modifica la ruta a api/catalogo, la prueba debe detectar que UltimaRuta ya no es /api/productos. Si modifica el contenido JSON para que nombre sea una propiedad diferente, debe investigar la correspondencia con ProductoDto.

Paso H4. Ejecutar sin servidor

Compila y ejecuta:

```
dotnet run
```

La salida esperada es «GET y deserialización correctos». Detén o mantén apagada la API del curso y ejecuta de nuevo: el resultado debe ser el mismo. El host laboratorio.local no está funcionando como un servidor real; el manejador está suministrando toda la respuesta.

La observación es importante: una comprobación de cliente HTTP no necesita siempre un servidor remoto. Sin embargo, tampoco demuestra por sí sola que una API exista o cumpla realmente el contrato. Es una prueba de un límite, no del sistema completo.

Paso H5. Provocar un cambio de ruta

Localiza la cadena api/productos en ObtenerAsync y sustitúyela temporalmente por api/catalogo. El manejador seguirá devolviendo una lista, porque en nuestra implementación de prueba no verifica por sí mismo la ruta; registra lo que recibe. La comprobación posterior debe detectar que UltimaRuta ya no coincide con el contrato esperado.

Esto permite explicar por qué una prueba necesita aserciones. Que un manejador de prueba devuelva 200 incondicionalmente no significa que la solicitud haya sido correcta. Tenemos que comprobar de manera explícita el verbo y la ruta, de lo contrario el programa podría aceptar una llamada distinta de la que pretendíamos construir.

Restaura api/productos antes de continuar.

Paso H6. Provocar una respuesta 404 controlada

En el manejador sustituye temporalmente la creación de la respuesta por:

```
var respuesta = new HttpResponseMessage(
    HttpStatusCode.NotFound)
{
    Content = new StringContent("Recurso inexistente")
};
return Task.FromResult(respuesta);
```

Si mantienes GetFromJsonAsync, la operación debe fallar por el estado HTTP no satisfactorio. El programa ya no imprimirá la confirmación de GET correcto. Este experimento enseña que el error no depende de una desconexión: el manejador ha devuelto una respuesta HTTP 404 de manera explícita.

Restablece después la respuesta 200 de la versión original. No crees un segundo método SendAsync con la misma firma dentro de la clase, porque el ejercicio utiliza una implementación alternativa del mismo comportamiento.

Paso H7. Provocar un JSON incompatible

Deja el estado HTTP en 200, pero cambia el contenido a:

```
{
  "cantidad": 1
}
```

El código ObtenerAsync sigue intentando convertir la respuesta a List. La respuesta puede ser JSON válido, pero no corresponde a una lista. Debe producirse un fallo de deserialización.

Esta prueba demuestra una segunda frontera: un estado 200 no garantiza que el contenido tenga la forma contratada. Restaurar la lista correcta soluciona el problema sin cambiar el método HTTP ni la dirección base.

Paso H8. Probar una colección vacía auténtica

Sustituye el cuerpo simulado por [] y adapta únicamente la comprobación de datos del programa para aceptar cero elementos. El método de deserialización debe devolver una lista vacía, que no es un error técnico.

Distingue esta situación de 404 y JSON incompatible. En el primer caso el servidor simulado ha respondido correctamente a una consulta con cero resultados. En el segundo ha rechazado la solicitud y en el tercero no cumplía el contrato de datos.

Vuelve después a la respuesta de Cuaderno y restaura las comprobaciones originales.

Paso H9. Conectar la prueba con IProductosApiService

El programa de consola era deliberadamente pequeño y contiene una función local ObtenerAsync. En el proyecto real, ProductosApiService utiliza un HttpClient suministrado por DI y ofrece operaciones mediante IProductosApiService.

La idea es equivalente: podemos pasar a esa implementación un HttpClient construido con un manejador de prueba, siempre que configuremos la dirección base apropiadamente, y observar el verbo y la ruta que genera. No hace falta copiar un framework de pruebas completo dentro del curso para comprender el principio.

El alumno debe explicar por qué es útil tener una capa dedicada a HTTP: el cliente puede examinarse por separado de Razor, y la UI puede utilizar una implementación alternativa de su interfaz. Son dos modalidades distintas de sustitución, cada una con una finalidad concreta.

Paso H10. Contrastar una prueba sin red con una integración real

Cuando hayas restaurado el programa, inicia la API de 4.3 en 5081 y consulta GET /api/productos con curl.exe o Visual Studio. La respuesta real también debe ser una lista compatible, pero ahora intervienen Kestrel, el enrutamiento de ASP.NET Core, el diccionario de productos y la serialización del servidor.

Compara las preguntas:

- La consola con manejador controlado comprueba qué solicitud construye el cliente y cómo interpreta un cuerpo conocido.
- La petición directa al servidor comprueba cómo responde la API para una entrada dada.

- La página Blazor prueba cómo colaboran componente, servicio y API durante una interacción real.

No intercambies esas evidencias como si fueran idénticas. Una prueba exitosa del cliente no corrige un endpoint inexistente, del mismo modo que un GET directo correcto no demuestra automáticamente que la pantalla muestre los datos.

P4.2 — Matriz de observaciones del manejador

Variante	Código HTTP	Cuerpo	Resultado previsto
Respuesta de referencia	200	Lista con Cuaderno	DTO correcto
Ruta cliente cambiada	200 simulado	Lista con Cuaderno	Falla la aserción de ruta
Recurso rechazado	404	Texto	Falla por estado HTTP
Estructura incompatible	200	Objeto resumen	Falla por deserialización
Catálogo vacío	200	[]	Lista vacía válida

Esta ampliación termina cuando el estudiante puede distinguir una prueba controlada del cliente HTTP de una prueba de integración con un servidor real. La distinción evita atribuir garantías que una comprobación nunca llegó a observar.

Consolidación

El resultado de P4.2 no es una pantalla nueva, sino una frontera de responsabilidades: el componente presenta y gestiona el estado; `IProductosApiService` expresa las operaciones que necesita; `ProductosApiService` resuelve el transporte; y la API mantiene las reglas del servidor. En P4.3 estudiaremos y modificaremos esta última.

P4.3 — Construir desde cero la API REST y conservar el contrato de Blazor

Objetivo y material inicial

Partimos de **la API entregada en 4.1**, `PROYECTO/4.1/CursoWeb.ProductosApi`, y hacemos una copia acumulativa en `PROYECTO/4.3/CursoWeb.ProductosApi`. Para la interfaz usamos el proyecto refactorizado de 4.2; lo conservaremos en `PROYECTO/4.3/CursoWeb.Blazor`. No se crea una API alternativa sin relación con lo aprendido.

Al terminar, la nueva API seguirá ofreciendo GET, POST, respuestas 400/404 y una operación adicional `GET /api/productos/resumen`. El cliente Blazor seguirá consultando `/api/productos` sin cambiar su código HTTP.

Recorrido obligatorio P4.3 — construir el servidor antes de estudiarlo como referencia

Sigue las **etapas 1 a 5** del taller de construcción autónoma: crear la plantilla ASP.NET Core, publicar GET, añadir detalle, implementar POST con validación e integrar la API recién creada con Blazor. La API entregada para P4.1–P4.2 solo era un servidor disponible para comenzar el consumo: **no se copia su Program.cs como solución del alumno**.

Al terminar la etapa 5, el contrato está demostrado. Las siguientes tareas —reinicio, resumen, código comentado del proyecto suministrado, búsqueda opcional, editor `.http` y PowerShell— constituyen ampliaciones. Pueden desarrollarse después de clase o utilizarse por el docente para enriquecer la demostración, sin interrumpir el avance hacia P4.4.

Construcción obligatoria — API REST a partir de una plantilla vacía

En 4.1 y 4.2 era indispensable disponer de una API operativa mientras aprendíamos a consumirla. Ahora podemos construir una nueva implementación desde cero y poner a prueba que **responde exactamente al mismo contrato HTTP**.

Preparación del escenario. Mantén el proyecto suministrado como referencia, sin editarlo. En otra carpeta de trabajo —por ejemplo, `PROYECTO/4.3/CursoWeb.ProductosApi.Construida`— crea una aplicación vacía con:

```
dotnet new web -n CursoWeb.ProductosApi -f net8.0
```

Detén la API de 4.1 antes de arrancar la nueva en el puerto 5081. Esta secuencia debe realizarse por etapas; después de cada una, compila y observa la respuesta para localizar los errores en el momento en que se introducen.

Etapa 1 — arranque y primer endpoint. Abre `Program.cs` y escribe:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => Results.Ok("API de Productos disponible"));

app.Run();
```

Ejecuta `dotnet run --urls http://localhost:5081` desde el directorio del proyecto. Al solicitar `http://localhost:5081/` se debe recibir 200 y un mensaje. Una respuesta positiva indica que hay un host operativo, pero todavía no que exista un catálogo de productos.

Etapa 2 — modelos y listado. Sustituye `Program.cs` por:

```
using System.Collections.Concurrent;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var productos = new ConcurrentDictionary<int, ProductoApi>();
productos[1] = new ProductoApi(1, "Cuaderno", 2.50m);
productos[2] = new ProductoApi(2, "Lápiz", 0.80m);

app.MapGet("/", () => Results.Ok("API de Productos disponible"));
app.MapGet("/api/productos", () =>
    Results.Ok(productos.Values.OrderBy(p => p.Id)));

app.Run();

record ProductoApi(int Id, string Nombre, decimal Precio);
```

Arranca de nuevo. El navegador debe mostrar dos objetos en un arreglo JSON cuando consultas `/api/productos`. Comprueba además que `/api/productos/1` todavía **no** devuelve un detalle válido: esa ruta aún no existe y esa ausencia es esperable antes de la etapa siguiente.

Etapa 3 — consulta individual. Antes de `app.Run()`, añade:

```
app.MapGet("/api/productos/{id:int}", (int id) =>
    productos.TryGetValue(id, out var producto)
        ? Results.Ok(producto)
        : Results.NotFound());
```

Reinicia y verifica `/api/productos/1`: debe aparecer Cuaderno. Después solicita `/api/productos/999` y observa 404. No confundas este resultado con un puerto cerrado: el servidor está respondiendo correctamente, pero el recurso no existe.

Etapa 4 — creación y validación. Antes de `app.Run()`, añade:

```
var siguienteId = 2;

app.MapPost("/api/productos", (CrearProductoApi entrada) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Nombre) || entrada.Precio <= 0)
        return Results.BadRequest("Nombre y precio deben ser válidos.");

    var id = Interlocked.Increment(ref siguienteId);
    var creado = new ProductoApi(id, entrada.Nombre.Trim(), entrada.Precio);
    productos[id] = creado;
    return Results.Created($"/api/productos/{id}", creado);
});
```

Al final del archivo, junto a `ProductoApi`, añade:

```
record CrearProductoApi(string Nombre, decimal Precio);
```

Prepara un archivo `nuevo-producto.json`:

```
{
  "nombre": "Regla",
  "precio": 1.75
}
```

Con la API iniciada ejecuta, desde una terminal donde el fichero sea accesible:

```
curl.exe -i -X POST http://localhost:5081/api/productos -H "Content-Type: application/json" --
data-binary "@nuevo-producto.json"
```

Debe obtenerse **201 Created**, un producto nuevo con ID asignado por el servidor y una cabecera `Location`. Repite GET del listado y comprueba que existe Regla. Cambia el precio del JSON a 0 y repite: debe devolverse **400 Bad Request**, sin añadir otro producto. Si vuelves a introducir datos válidos, el contador puede seguir creciendo; es normal mientras no se reinicie el proceso.

Etapa 5 — integración con el cliente Blazor que ya construimos. Mantén la API recién creada en 5081 y abre Blazor desde `PROYECTO/4.2/CursoWeb.Blazor`. Consulta `/catalogo-remoto`. Si el contrato se ha conservado, verás los productos sin reescribir `CatalogoRemoto.razor` ni `ProductosApiService.cs`. Envía un nuevo producto desde la interfaz y comprueba el 201 indirectamente observando que se añade al listado.

Para diagnosticar un fallo, recorre en orden estas preguntas: ¿responde la API directamente?, ¿coinciden la ruta y el puerto?, ¿el JSON conserva `id`, `nombre` y `precio`?, ¿la clase `CrearProductoApi` admite los datos enviados?, ¿la página recibe un error HTTP o un error de red?

Etapa 6 — repetición controlada y persistencia. Detén y vuelve a iniciar el servidor. La lista debe regresar a Cuaderno y Lápiz porque el almacenamiento era de memoria. Este comportamiento no supone un defecto oculto: es un límite expresamente elegido para concentrarse en comunicación HTTP. No incorpores EF Core en esta etapa.

Reto de sustitución del servidor. Sin modificar el cliente, cambia temporalmente la cadena `"Cuaderno"` de la API construida por `"Cuaderno A4"`, reinicia y consulta Blazor. El nombre debe reflejar el cambio de servidor. Después restaura el dato de referencia y explica por qué la interfaz no necesitó recompilarse para recibir una respuesta JSON con el mismo contrato.

Ampliación P4.3 — comparar con la API de referencia y activar el resumen

Este apartado es posterior a la construcción autónoma. Ahora resulta útil leer la versión de referencia y comparar sus endpoints; **no** sustituye los pasos anteriores.

Paso 4.3.1. Identificar las piezas del proceso servidor

Abre `CursoWeb.ProductosApi/Program.cs` e identifica el `WebApplication.CreateBuilder`, `Build`, `MapGet`, `MapPost` y `Run`. Localiza los registros Cuaderno y Lápiz que se cargan al arrancar, y observa que se guardan en `ConcurrentDictionary<int, ProductoApi>`, no en una base de datos.

Ejecuta la API:

```
dotnet run --project M04_APIS_SEGURIDAD/PROYECTO/4.3/CursoWeb.ProductosApi/
CursoWeb.ProductosApi.csproj --urls http://localhost:5081
```

Si otra instancia usa 5081, detén esa instancia antes de iniciar la de 4.3. Abre `http://localhost:5081/api/productos` y verifica que aparecen dos productos.

Paso 4.3.2. Seguir un GET y un POST

En el código, localiza `Results.Ok` y `Results.Created`. Consulta `/api/productos/999` para obtener 404. Envía un POST con nombre «Regla» y precio 1,75 usando un cliente HTTP (por ejemplo, `curl.exe` con un fichero JSON para evitar problemas de comillas):

```
{ "nombre": "Regla", "precio": 1.75 }
```

Observa la respuesta 201, el identificador asignado y la cabecera `Location`. Envía ahora un nombre vacío y precio cero para obtener 400. Comprueba que un producto inválido no aparece en la lista GET.

Paso 4.3.3. Añadir una ruta de resumen

Antes del `app.Run();` añade este endpoint, cuya finalidad y contrato se explican en teoría 4.3.4:

```
// PASO 4.3.3 — activar al llegar aquí.
app.MapGet("/api/productos/resumen", () =>
{
    var actuales = productos.Values.ToArray();
    return Results.Ok(new
    {
        cantidad = actuales.Length,
        precioTotal = actuales.Sum(p => p.Precio)
    });
});
```

El proyecto de laboratorio incluye el bloque **comentado** con marca `PASO 4.3.3`, para que puedas descomentarlo antes de ejecutarlo. Tras arrancar desde datos iniciales, `GET /api/productos/resumen` debe devolver `cantidad` 2 y `precioTotal` 3,30. Crea Regla y vuelve a consultar: los valores deberían ser 3 y 5,05. Si antes añadiste otros productos, reinicia el proceso para reproducir el estado de partida.

Paso 4.3.4. Conectar la API ampliada con la pantalla del paso 4.2

Ejecuta `CursoWeb.Blazor` del punto 4.3 en otro puerto; la API permanece en 5081. Abre `/catalogo-remoto` y crea un producto. Comprueba que sigue apareciendo en el listado y que `/api/productos/resumen` refleja la creación **si has activado** el bloque del paso anterior. Observa que no hemos tenido que editar el `IProductosApiService`: el nuevo endpoint es una capacidad del servidor que no interfiere con el contrato utilizado por el cliente.

Paso 4.3.5. Reto: un segundo criterio de resumen

A partir del endpoint de resumen, añade la propiedad `hayProductos` con valor booleano:

```
hayProductos = actuales.Length > 0
```

La variante está incluida como bloque `RETO 4.3.5` comentado en el proyecto. Para activarla, reemplaza el bloque de `PASO 4.3.3` por el bloque completo del reto: **no actives ambas versiones simultáneamente**, porque intentarían asignar la misma ruta GET.

Haz otra llamada a `/api/productos/resumen` y comprueba que aparece `"hayProductos": true`. Explica por qué es un campo de un JSON distinto de `ProductoDto` y por qué el cliente de 4.2 no necesita cambios.

Ampliación P4.3 — Registro detallado de la construcción: del host vacío a una API compatible

La construcción de la API debe realizarse por etapas porque cada etapa comprueba una decisión específica. Mantén abierta una terminal para el proceso servidor y otra para enviar peticiones. No incorpores EF Core ni utilices una base de datos: queremos que la evidencia de cada estado corresponda a un endpoint, una ruta o una regla HTTP.

Estado A: el host. Después de crear la plantilla web, Program.cs configura la aplicación y publica una ruta raíz de bienvenida. Compila y arranca. Si el proceso informa de que el puerto está ocupado, detén la API suministrada: no modifiques Blazor para resolver un conflicto de puertos del servidor.

Estado B: GET del catálogo. Introduce el modelo ProductoApi y los dos productos iniciales. Registra GET /api/productos y comprueba que devuelve un arreglo JSON con Cuaderno y Lápiz. No confundas la respuesta de la raíz con el catálogo: son endpoints distintos. Si GET /api/productos devuelve 404, revisa la ruta realmente registrada.

Estado C: detalle por ID. Añade GET /api/productos/{id:int}. Ejecuta una consulta con ID 1 y comprueba que devuelve Cuaderno; después solicita ID 999 y comprueba 404. En este punto el código del endpoint puede ser correcto aunque la colección no incluya el ID solicitado. No interpretes un 404 como prueba de que el servidor está apagado.

Estado D: DTO de creación. Declara CrearProductoApi con Nombre y Precio, sin Id. El cliente que crea un recurso todavía no conoce el identificador asignado. El enlace JSON debe permitir que el servidor construya el objeto de entrada a partir del cuerpo de POST.

Estado E: validación. Antes de crear el recurso, comprueba que Nombre contiene texto significativo y que Precio es mayor que cero. Una entrada inválida produce 400 y no modifica la colección. Ejecuta un POST inválido y comprueba que el siguiente GET conserva el mismo número de objetos que antes.

Estado F: asignación de identidad. Utiliza un contador compartido inicializado en 2 e Interlocked.Increment para obtener el identificador de una nueva creación. El producto devuelto debe contener ese ID real, que también se utiliza al formar la dirección Location. La colección temporal se actualiza con el objeto creado y el servidor responde 201.

Estado G: compatibilidad con Blazor. Detén la API suministrada e inicia la API creada por el alumno en el mismo puerto. Abre /catalogo-remoto. Si los dos servidores cumplen el contrato, no necesitas reescribir el componente ni ProductosApiService. Si falla, compara las URL y los campos JSON antes de cambiar código de UI.

Estado H: resumen. Después de dominar el contrato base, añade GET /api/productos/resumen. Debe devolver cantidad y precioTotal calculados a partir del contenido actual. No modifiques GET /api/productos: el cliente continúa esperando una lista, no un objeto resumen.

Estado I: variante de reto. Sustituye temporalmente el endpoint de resumen por la variante que añade hayProductos. No registres simultáneamente las dos versiones del mismo método y ruta. Comprueba el JSON y restablece el estado base si vas a continuar con los ejemplos de otros apartados.

Código del proyecto suministrado: guía de correspondencia con la API creada

Este es el Program.cs que acompaña al ejercicio. La API de 4.1 funciona desde el inicio para que los ejercicios de consumo no dependan de P4.3. En 4.3 puede compararse con el servidor reconstruido y localizar la equivalencia de sus endpoints. Los comentarios identifican el paso y el reto, que se activan por separado.

```
using System.Collections.Concurrent;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// P4.1: API de laboratorio disponible antes de comenzar el cliente.
// En P4.3 se explicará y modificará este mismo proyecto.
var productos = new ConcurrentDictionary<int, ProductoApi>();
productos[1] = new ProductoApi(1, "Cuaderno", 2.50m);
productos[2] = new ProductoApi(2, "Lápiz", 0.80m);
var siguienteId = 2;

// GET 200 + catálogo JSON.
app.MapGet("/api/productos", () =>
    Results.Ok(productos.Values.OrderBy(p => p.Id)));

// POST 400 para entradas inválidas; 201 y recurso creado si es válido.
app.MapPost("/api/productos", (CrearProductoApi entrada) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Nombre) || entrada.Precio <= 0)
        return Results.BadRequest("Nombre y precio deben ser válidos.");

    var id = Interlocked.Increment(ref siguienteId);
    var creado = new ProductoApi(id, entrada.Nombre.Trim(), entrada.Precio);
    productos[id] = creado;
    return Results.Created($"/api/productos/{id}", creado);
});

// GET 404 sirve como ejemplo de respuesta no satisfactoria.
app.MapGet("/api/productos/{id:int}", (int id) =>
    productos.TryGetValue(id, out var producto)
        ? Results.Ok(producto)
        : Results.NotFound());


// PASO 4.3.3 INICIO — quitar los delimitadores de comentario para activar la ruta.
/*
app.MapGet("/api/productos/resumen", () =>
{
    var actuales = productos.Values.ToArray();
    return Results.Ok(new { cantidad = actuales.Length, precioTotal = actuales.Sum(p =>
p.Precio) });
});
*/
// PASO 4.3.3 FIN

// RETO 4.3.5 INICIO — variante alternativa; no activar junto al paso 4.3.3.
/*
app.MapGet("/api/productos/resumen", () =>
{
    var actuales = productos.Values.ToArray();
    return Results.Ok(new
    {
        cantidad = actuales.Length,
        precioTotal = actuales.Sum(p => p.Precio),
```

```
        hayProductos = actuales.Length > 0
    });
});
*/
// RETO 4.3.5 FIN

app.Run();

record ProductoApi(int Id, string Nombre, decimal Precio);
record CrearProductoApi(string Nombre, decimal Precio);
```

Banco de pruebas HTTP reproducible

Prueba 1 — Catálogo inicial

Arranca la API sin haber creado productos adicionales y consulta GET /api/productos. La respuesta debe ser una colección de dos elementos. Registra sus identificadores y precios. Si observas más elementos, comprueba si estás reutilizando un proceso que ya recibió POST; reiniciarlo restaura el estado inicial de memoria.

Prueba 2 — Identificador existente

Consulta GET /api/productos/1. Comprueba que el JSON contiene id 1 y nombre Cuaderno. Esto demuestra que la plantilla de ruta y la lectura del diccionario están funcionando juntas.

Prueba 3 — Identificador inexistente

Consulta GET /api/productos/999. Comprueba que el servidor responde 404. El criterio de éxito de esta prueba no es recibir un objeto; es que un recurso inexistente se representa como no encontrado y no como error interno 500.

Prueba 4 — Cuerpo de creación correcto

Prepara nuevo-producto.json con nombre Regla y precio 1.75, y envíalo con curl.exe usando Content-Type application/json. La respuesta debe ser 201 y contener un producto con un ID nuevo. Comprueba la cabecera Location, que proporciona la dirección del recurso. Si el ID no coincide con el esperado en una ejecución limpia, observa si ha habido altas previas.

Prueba 5 — GET después de POST

Consulta GET /api/productos inmediatamente después del alta. La colección debe contener Regla. El servidor conserva los productos durante la vida del proceso, sin intervención de la página Blazor. Si no aparece, comprueba que GET y POST consultan el mismo diccionario de memoria y no listas creadas por separado.

Prueba 6 — Nombre vacío

Envía un cuerpo válido como JSON pero con nombre vacío. Se espera 400. Comprueba después la colección: no debe haber aumentado. Si aumenta, la validación está situada demasiado tarde o no cubre la regla definida.

Prueba 7 — Solo espacios en el nombre

Envía un nombre compuesto únicamente por espacios. También debe ser rechazado, porque string.IsNullOrWhiteSpace contempla el caso. Si se acepta, revisa que la comprobación de la API utiliza el método esperado y no solo una comparación con cadena vacía.

Prueba 8 — Precio cero

Envía un precio igual a cero. Se espera 400. Esta prueba es independiente de la validación previa de Blazor; puedes enviarla mediante una herramienta HTTP directa aunque el formulario de la página no permita hacerlo.

Prueba 9 — Precio negativo

Envía un valor menor que cero. Se espera 400. No es necesario que el nombre sea inválido: cada caso debe aislar una condición de rechazo. Cuando se prueban varias entradas inválidas simultáneamente, cuesta saber cuál de las reglas respondió.

Prueba 10 — Nombre con espacios alrededor

Envía un nombre con espacios al principio y al final, manteniendo un precio válido. La API utiliza Trim y devuelve el nombre sin esos espacios. Consulta el producto individual y comprueba que conserva el valor normalizado.

Prueba 11 — Ruta inexistente

Consulta una ruta no registrada, como GET /api/catalogo. Se espera 404. Esto no es lo mismo que solicitar un producto ID 999, pero ambos escenarios pueden producir el mismo estado protocolario: uno se debe al enrutamiento y el otro a la lógica del recurso.

Prueba 12 — URL del resumen antes de activarlo

Consulta GET /api/productos/resumen antes de añadir el endpoint. La ruta no está disponible. Esta observación representa el punto de partida del paso 4.3.3: cuando actives el código, habrá un cambio observable y no solo un bloque de código escrito.

Prueba 13 — Resumen después de activarlo

Activa exclusivamente el bloque PASO 4.3.3. Reinicia y consulta el resumen. Con los dos productos iniciales, cantidad debe ser 2 y precioTotal 3.30. La forma del cuerpo debe ser un objeto, no una lista de productos.

Prueba 14 — Recalcular tras el alta

Con el resumen activo, crea Regla por 1.75 y vuelve a consultarlo. La cantidad debe pasar a 3 y el precio total a 5.05. Así compruebas que el endpoint calcula a partir del diccionario actual y no muestra números fijos.

Prueba 15 — Variante hayProductos

Desactiva el bloque PASO 4.3.3 y activa RETO 4.3.5. Vuelve a compilar y consulta el resumen. Debe incluir hayProductos con valor true mientras la colección tenga productos. No mantengas dos endpoints GET para la misma ruta, porque el objetivo del reto es modificar un contrato de salida y no provocar ambigüedad de endpoints.

Prueba 16 — Persistencia intencionadamente ausente

Crea un producto y verifica que aparece en GET. Detén completamente la API y arráncala de nuevo. El catálogo debe regresar a Cuaderno y Lápiz. La ausencia de persistencia no es un fallo del código: es un límite conocido para este ejercicio de HTTP y memoria.

Prueba 17 — Compatibilidad sin recompilar Blazor

Ejecuta la API construida desde cero en el mismo puerto y abre la aplicación Blazor de P4.2. Consulta y crea productos. El cliente debe seguir funcionando. Si no lo hace, compara la respuesta JSON y la ruta que utiliza el servicio. En este laboratorio, sustituir el servidor conservando el contrato no debe requerir modificar el componente Razor.

Tabla de evidencias de P4.3

Operación	Petición	Estado esperado	Recurso final
Consultar listado	GET /api/productos	200	Colección
Consultar ID 1	GET /api/productos/1	200	Objeto
Consultar ID 999	GET /api/productos/999	404	No encontrado
Crear válido	POST /api/productos	201	Objeto con nuevo ID
Crear nombre vacío	POST /api/productos	400	No se crea
Crear precio cero	POST /api/productos	400	No se crea
Resumen activado	GET /api/productos/resumen	200	Cantidad y precioTotal
Resumen con reto	GET /api/productos/resumen	200	Incluye hayProductos
Proceso reiniciado	GET /api/productos	200	Dos elementos iniciales

Esta tabla permite evaluar el contrato sin confundir éxito de compilación con éxito funcional. Es recomendable escribir el resultado observado junto a cada prueba cuando el alumno la ejecuta, aunque no incorporaremos un sistema de auditoría a su código fuente.

Ampliación P4.3 — Laboratorio de solicitudes en Visual Studio

Además de consultar la API desde Blazor, vamos a ejecutarla como lo haría otro consumidor HTTP. Visual Studio puede abrir archivos .http que contienen solicitudes separadas mediante líneas ###. Cada solicitud declara el verbo, la ruta, sus cabeceras y, cuando es necesario, un cuerpo JSON. La utilidad pedagógica de este archivo es poder reproducir las operaciones sin escribir un programa cliente adicional.

El proyecto de referencia incorpora ProductosApi.http. También puedes crearlo manualmente dentro de la carpeta de la API que hayas construido desde cero. La variable BaseUrl concentra el host y el puerto. Si has arrancado una API diferente en otra dirección, cambia ese valor en una copia del fichero y no modifiques todos los endpoints uno a uno.

Preparación del laboratorio HTTP

Antes de comenzar, detén los procesos que puedan estar usando 5081. Inicia exclusivamente la API que deseas comprobar; por ejemplo, la que has construido mediante dotnet new web. Observa en

la consola que ha comenzado a escuchar. Comprueba con el navegador que GET /api/productos responde, y después abre el editor HTTP.

No ejecutes de una sola vez todas las solicitudes del archivo suponiendo que los resultados serán independientes. Algunas, como POST Regla y POST Agenda, modifican el diccionario en memoria. El identificador de una creación depende del estado que ya exista en ese proceso. Para observar exactamente el caso de dos elementos iniciales, reinicia antes la API.

Solicitudes ejecutables del laboratorio

El contenido del archivo es el siguiente:

```
# M04 · P4.3 · Peticiones que puede ejecutar el alumno en Visual Studio
# Arrancar antes la API en http://localhost:5081
# Los resultados de POST dependen del estado temporal del proceso.
@BaseUrl = http://localhost:5081

### A. Catálogo inicial: 200 y colección de dos elementos
GET {{BaseUrl}}/api/productos
Accept: application/json

### B. Detalle existente: 200, Cuaderno
GET {{BaseUrl}}/api/productos/1
Accept: application/json

### C. Identificador inexistente: 404
GET {{BaseUrl}}/api/productos/999
Accept: application/json

### D. Ruta inexistente: 404 por enrutamiento
GET {{BaseUrl}}/api/catalogo
Accept: application/json

### E. Creación correcta: 201 + Location
POST {{BaseUrl}}/api/productos
Content-Type: application/json
Accept: application/json

{
  "nombre": "Regla",
  "precio": 1.75
}

### F. Revisar el catálogo tras la creación
GET {{BaseUrl}}/api/productos
Accept: application/json

### G. Rechazo de precio cero: 400
POST {{BaseUrl}}/api/productos
Content-Type: application/json

{
  "nombre": "Regla",
  "precio": 0
}

### H. Rechazo de nombre vacío: 400
POST {{BaseUrl}}/api/productos
Content-Type: application/json

{
  "nombre": "",
```

```
  "precio": 1.75
}

### I. Rechazo de nombre formado por espacios: 400
POST {{BaseUrl}}/api/productos
Content-Type: application/json

{
  "nombre": "   ",
  "precio": 1.75
}

### J. Rechazo de precio negativo: 400
POST {{BaseUrl}}/api/productos
Content-Type: application/json

{
  "nombre": "Regla",
  "precio": -0.01
}

### K. Nombre con espacios a normalizar: 201
POST {{BaseUrl}}/api/productos
Content-Type: application/json

{
  "nombre": "   Agenda   ",
  "precio": 6.40
}

### L. Resumen (ACTIVAR PASO 4.3.3): 200, cantidad y precioTotal
GET {{BaseUrl}}/api/productos/resumen
Accept: application/json

### M. Resumen extendido (ACTIVAR RETO 4.3.5 EN VEZ DEL PASO):
# El resultado incluye hayProductos; no activar ambas rutas a la vez.
GET {{BaseUrl}}/api/productos/resumen
Accept: application/json
```

Interpretación 1: distinguir las dos clases de 404

Ejecuta primero la solicitud de detalle con ID 999. El endpoint GET /api/productos/{id:int} existe, pero el diccionario no contiene ese identificador y la lógica devuelve NotFound. Ahora ejecuta la solicitud de /api/catalogo. En este segundo caso no hemos publicado una ruta que corresponda a esa petición.

Aunque ambas operaciones pueden responder 404, sus causas son diferentes. La primera demuestra que la lógica de un recurso controla una ausencia válida; la segunda que la ruta solicitada no coincide con un endpoint disponible. Saber diferenciarlas evita corregir datos cuando el problema es una dirección equivocada.

Interpretación 2: cuerpo de creación y estado HTTP

Ejecuta la solicitud POST correcta de Regla. Comprueba que el servidor recibe JSON con nombre y precio, que responde con 201 y que entrega el recurso nuevo. Observa si aparece Location y qué dirección señala. Después ejecuta el GET del catálogo y verifica que el servidor realmente incorporó el elemento.

Una respuesta 201 sin posibilidad de consultar el recurso o sin el producto esperado merece investigación: la creación no queda demostrada solamente porque haya aparecido una cifra en un

encabezado de respuesta. En este laboratorio, la evidencia conjunta es el POST, el producto devuelto y su aparición posterior en GET.

Interpretación 3: reglas de negocio frente a sintaxis JSON

El archivo incluye casos de nombre vacío, nombre compuesto por espacios, precio cero y precio negativo. Todos son JSON sintácticamente válido. El servidor debe convertir esos cuerpos a CrearProductoApi y después rechazarlos por las reglas de negocio del ejercicio.

Una entrada con JSON mal formado es un caso distinto: el problema puede aparecer durante la lectura del cuerpo antes de ejecutar las reglas de validación. El alumno no debe atribuir a la función de comprobación del nombre todos los errores que se producen en una solicitud POST.

Al ejecutar cada petición inválida, realiza GET inmediatamente después y verifica que la colección no ha aumentado. No basta con observar 400; también queremos descartar que se haya añadido el elemento antes de rechazarlo.

Interpretación 4: normalización de datos

Ejecuta el POST con espacios antes y después del nombre Agenda. El servidor llama Trim antes de crear el producto, por lo que la respuesta debe contener el nombre normalizado. La UI de Blazor también aplica una normalización básica antes de enviar. Estas dos decisiones no son redundantes de manera accidental: la primera ayuda al usuario y la segunda protege el contrato del servidor frente a cualquier cliente HTTP.

Explica por qué el servidor no debe confiar en que todos los consumidores utilizarán necesariamente el formulario que hemos construido. La existencia del fichero .http es una prueba de que otro cliente puede invocar la operación sin pasar por el componente Razor.

Interpretación 5: el resumen es una operación distinta

Antes de activar el endpoint de resumen, su solicitud no debe devolver el objeto esperado. Ahora activa exclusivamente PASO 4.3.3 en Program.cs, compila y reinicia. Consulta de nuevo el resumen y comprueba que incluye cantidad y precioTotal.

Si se habían creado productos antes de reiniciar, recuerda que la colección vuelve al estado inicial de memoria. Para comprobar el valor después de un alta, realiza POST Regla y a continuación solicita de nuevo el resumen. Así el resultado debe reflejar tres productos y la suma de sus precios.

Después desactiva el paso normal y activa la variante RETO 4.3.5. El JSON debe incorporar hayProductos. No actives ambos bloques simultáneamente: publicarías dos GET con la misma ruta, lo que dificulta seleccionar un único resultado y contradice el propósito de la variante.

Interpretación 6: el cliente Blazor conserva su contrato

Una vez que has comprobado directamente la API construida por el alumno, deja ese proceso escuchando en 5081 y arranca el proyecto Blazor de 4.2. La página /catalogo-remoto debe cargar la lista y permitir crear un producto.

Comprueba en ProductosApiService.cs que las rutas siguen siendo api/productos y que el servicio utiliza DTO de lectura y creación compatibles. Si el .http funciona pero el cliente Blazor no, ya has aislado una parte del problema: revisa la configuración del cliente HTTP y la serialización antes de reescribir el endpoint.

Si el servidor devuelve un objeto de resumen correcto pero Blazor no lo presenta en su catálogo, no es un fallo: esa pantalla consume deliberadamente la lista original. Añadir una pantalla específica para el resumen sería otro requisito, no un cambio implícito de GET /api/productos.

P4.3 — Ficha individual para estudiar respuestas

Para cada operación ejecutada anota mentalmente o en tus notas de trabajo cinco datos: método y ruta, cuerpo de solicitud, estado obtenido, forma del cuerpo de respuesta y cambio de estado del servidor si existe. Esa pauta impide clasificar una operación por una sola cifra.

Por ejemplo, para POST Regla: el método es POST, la ruta /api/productos, la entrada contiene nombre y precio, el estado esperado es 201, el cuerpo es un objeto con ID y la colección se incrementa. Para GET /api/productos/999: método GET, sin cuerpo de creación, estado 404 y sin modificación del diccionario.

La pauta cambia al examinar un problema de conectividad. Si el servidor está apagado, no hay estado HTTP emitido por la API; por tanto, describirlo simplemente como un 404 sería incorrecto. Esa distinción se ha practicado también en P4.1, ahora desde el otro extremo de la comunicación.

P4.3 — Plan de recuperación ante errores de construcción

El proyecto no compila después de añadir los records. Comprueba que las instrucciones de nivel superior aparezcan antes de las declaraciones de tipos auxiliares. Revisa que los records finales se sitúen fuera del cuerpo de otro bloque y no dentro del lambda de un endpoint.

La API arranca pero la raíz devuelve un resultado diferente. Comprueba qué plantilla dotnet new utilizaste y qué rutas añadiste. No presupongas que la plantilla web vacía publica automáticamente /api/productos. Ese endpoint pertenece al código que el alumno debe construir.

El GET funciona pero el POST responde 404. Observa qué verbos se han registrado para /api/productos. Tener MapGet no implica que exista MapPost. Una ruta puede reconocer distintas operaciones según el método HTTP.

El POST responde 400 para un precio que parece válido. Revisa el cuerpo JSON real que envía la herramienta, la representación decimal y el nombre del campo. No uses una coma decimal fuera de una cadena en JSON: el formato numérico JSON emplea el punto. Después comprueba la regla de precio positivo.

El POST devuelve 201 pero no localizas el producto. Comprueba que GET y POST comparten la misma instancia del diccionario y que la API no se ha reiniciado entre las dos solicitudes. La memoria de laboratorio vive únicamente mientras el proceso está en ejecución.

El resumen devuelve una lista. Puede que sigas consultando /api/productos en lugar de /api/productos/resumen. Comprueba la URL efectiva antes de modificar los DTO. Cada endpoint tiene su propio contrato.

El resumen falla tras activar el reto. Comprueba que has desactivado el bloque PASO 4.3.3. Solo se debe registrar una de las variantes del mismo método y ruta. El resultado debe ser un objeto único con las propiedades del contrato extendido.

P4.3 — Conclusión que el alumno debe poder demostrar

Conserva dos proyectos diferentes: el cliente Blazor y la API que has construido desde cero. Pon la API en marcha, comprueba sus endpoints con el archivo .http y después inicia Blazor. El catálogo

remoto debe funcionar sin cambiar el código del componente y el servidor debe seguir rechazando las entradas inválidas incluso cuando no interviene la interfaz.

Si puedes explicar por qué esas tres condiciones son simultáneamente necesarias —funcionamiento directo del servidor, compatibilidad del cliente y validación independiente— habrás comprendido el propósito de crear la API dentro de M04.

Ampliación P4.3 — Reto opcional 4.3.6: incorporar una búsqueda sin romper los clientes existentes

Este ejercicio amplía la API que has construido desde cero con una operación de consulta que filtra productos por una parte de su nombre. No hace falta modificar el cliente de 4.2 ni reconstruir ninguna pantalla; vamos a probar el endpoint directamente y comprender cómo ASP.NET Core obtiene un parámetro desde la URL.

El resultado esperado es una nueva dirección: GET /api/productos/buscar?nombre=cuad. Con los productos iniciales, debe devolver Cuaderno dentro de un arreglo JSON. El servidor no debe modificar su diccionario al procesar esa petición y el cliente Blazor debe seguir consumiendo normalmente GET /api/productos.

Etapa B1. Establecer el estado de partida

Inicia la API en 5081 sin activar todavía el nuevo endpoint. Consulta GET /api/productos y comprueba que devuelve dos objetos. Intenta después solicitar /api/productos/buscar?nombre=cuad: antes de registrar la nueva ruta no se obtiene la lista filtrada. Anota esta observación porque será la referencia para demostrar que la ampliación tuvo un efecto real.

Asegúrate de que los bloques PASO 4.3.3 y RETO 4.3.5 estén en el estado que necesita tu ejercicio anterior. El endpoint de búsqueda utiliza una ruta nueva, de modo que puede coexistir con cualquiera de las versiones del resumen, pero no requiere tener dos resúmenes activados simultáneamente.

Etapa B2. Reconocer un parámetro de consulta

Descompón la dirección en dos partes: /api/productos/buscar identifica el endpoint y ?nombre=cuad transmite el criterio. No confundas ese parámetro con el id entero de /api/productos/{id:int}. El primero es un texto de búsqueda; el segundo selecciona un producto concreto por identificador.

La firma del manejador utiliza string? nombre. La posibilidad de null permite representar que el cliente no ha proporcionado un término. La primera condición del endpoint lo rechaza mediante BadRequest para que un GET sin criterio no se interprete silenciosamente como una consulta de toda la colección.

Etapa B3. Activar el endpoint

Localiza en Program.cs el bloque marcado RETO OPCIONAL 4.3.6. Retira solamente los delimitadores de comentario de ese bloque. No copies el código en otro lugar ni publiques la misma ruta por duplicado.

El contenido activado debe ser equivalente a:

```
app.MapGet("/api/productos/buscar", (string? nombre) =>
{
```

```
    if (string.IsNullOrWhiteSpace(nombre))
        return Results.BadRequest("Indica el nombre que deseas buscar.");

    var termino = nombre.Trim();
    var encontrados = productos.Values
        .Where(p => p.Nombre.Contains(
            termino, StringComparison.OrdinalIgnoreCase))
        .OrderBy(p => p.Id)
        .ToArray();

    return Results.Ok(encontrados);
});
```

Compila antes de ejecutar. Si falla, verifica que has quitado los dos delimitadores del mismo bloque y que el código quedó fuera de otro lambda. Reinicia la API para que el proceso ejecute la versión nueva.

Etapa B4. Búsqueda con una coincidencia

En ProductosApi.http ejecuta el bloque correspondiente a GET /api/productos/buscar?nombre=cuad. Debe devolver un arreglo con Cuaderno y un estado HTTP 200.

Comprueba que la respuesta sigue utilizando los mismos campos de ProductoApi: id, nombre y precio. No hemos definido un DTO de resultado diferente para la búsqueda, porque continúa devolviendo productos. La forma de lista permite que un cliente que conozca ProductoDto pueda interpretar el cuerpo, aunque el componente obligatorio del curso todavía no consuma esta nueva ruta.

Etapa B5. Búsqueda sin coincidencias

Ejecuta GET /api/productos/buscar?nombre=nohaycoincidencia. El resultado esperado es 200 con []. La solicitud es válida y el filtro se ha ejecutado, pero ningún nombre contiene ese término.

Es importante no cambiar la implementación para devolver 404 simplemente porque no haya coincidencias. En este contrato el cliente está consultando una colección filtrada; que esa colección resulte vacía es un resultado válido. El contraste con GET /api/productos/999 muestra por qué un recurso individual ausente y una colección sin resultados no tienen necesariamente el mismo estado HTTP.

Etapa B6. Criterio ausente

Ejecuta GET /api/productos/buscar sin la parte ?nombre=.... El endpoint rechaza la operación con 400. Comprueba que la respuesta corresponde al manejador de búsqueda y no a una ruta diferente.

Ahora prueba ?nombre= seguido de una cadena vacía. El comportamiento también debe ser 400. La condición string.IsNullOrWhiteSpace cubre ambos casos y permite expresar de manera sencilla la regla de que una búsqueda debe tener un término significativo.

Etapa B7. Mayúsculas y coincidencia parcial

Solicita ?nombre=CUAD y después ?nombre=cuad. Ambos filtros deben encontrar Cuaderno, porque el endpoint utiliza StringComparison.OrdinalIgnoreCase. El alumno debe localizar esta decisión en el código y explicar que no depende de cambiar la capitalización del nombre almacenado.

Después prueba un fragmento que no aparezca en ningún nombre. La respuesta vacía no debe provocar una excepción de conversión JSON, porque sigue siendo un arreglo correcto del tipo esperado.

Etapa B8. Búsqueda después de crear otro producto

Utiliza POST /api/productos para crear un producto llamado Cuaderno grande, con precio positivo. Repite la búsqueda por cuad. Ahora deben aparecer Cuaderno y Cuaderno grande, ordenados por identificador.

Esta secuencia demuestra que el servidor no utiliza una lista fija de resultados de búsqueda. Consulta el diccionario compartido que también utiliza el endpoint de creación. Si al repetir GET solo aparece el primer producto, investiga si el POST realmente se ejecutó en el mismo proceso que está atendiendo las consultas.

Etapa B9. La API original no debe haber cambiado

Solicita GET /api/productos sin filtro. Debe continuar devolviendo todos los elementos, incluidos los creados durante la ejecución. Después solicita el detalle por ID y realiza un POST válido. Estas operaciones no deben haber cambiado por añadir la búsqueda.

Este último paso es una comprobación de compatibilidad. El reto no se considera correcto solo porque la ruta nueva funcione: también debe conservar los contratos que necesitaba el cliente Blazor.

Etapa B10. Integración conceptual con el servicio tipado

El ejercicio obligatorio se detiene en la comprobación directa del endpoint. Como actividad de razonamiento, explica qué cambios serían necesarios para incorporar una caja de búsqueda remota a Blazor. Habría que añadir a IProductosApiService una operación BuscarAsync, implementarla en ProductosApiService y crear o adaptar la interfaz que recoge el criterio.

Además, el criterio introducido por un usuario debe representarse adecuadamente en la URL. No conviene construir rutas mediante concatenación ingenua de cadenas que puedan contener espacios o caracteres especiales. Este tema puede tratarse como ampliación de lectura, sin añadir más pantallas obligatorias a la jornada.

Etapa B11. Restaurar o conservar el reto de forma identificable

Cuando termines, puedes volver a comentar el bloque para conservar la base del ejercicio o dejarlo activado en una copia identificada como variante. No edites el proyecto de 4.1 que proporcionaba la API inicial y no elimines los endpoints originales de listado, detalle y creación.

La enseñanza del reto es concreta: diferenciar ruta y query string, validar un criterio, filtrar una colección sin modificarla, reconocer respuestas vacías legítimas y ampliar un contrato de API conservando su compatibilidad.

P4.3 — Evidencia final de la búsqueda

Solicitud	Estado esperado	Resultado
/api/productos/buscar?nombre=cuad	200	Cuaderno
/api/productos/buscar?nombre=CUAD	200	Cuaderno

Solicitud	Estado esperado	Resultado
/api/productos/buscar?nombre=nohaycoincidencia	200	[]
/api/productos/buscar sin nombre	400	Criterio rechazado
/api/productos/buscar?nombre=	400	Criterio rechazado
Búsqueda tras crear Cuaderno grande	200	Dos productos coincidentes
/api/productos habitual	200	Catálogo completo

No confundas esta matriz con una obligación de implantar búsquedas complejas, paginación o repositorios. Es una ampliación limitada de Minimal API que conecta un concepto de HTTP con un comportamiento que puede comprobarse directamente.

Ampliación P4.3 — Alternativa PowerShell: enviar JSON sin errores de comillas

Los comandos curl.exe de la práctica permiten observar las cabeceras y el estado HTTP, pero en Windows PowerShell puede resultar incómodo escribir JSON directamente en una línea de comandos que contiene varias comillas. Una alternativa consiste en preparar primero un objeto de PowerShell, convertirlo a JSON y enviarlo con los comandos HTTP integrados en ese entorno.

Este apartado no sustituye el archivo ProductosApi.http ni cambia la API. Ofrece otra forma de ejecutar los mismos contratos con los conocimientos que el alumno ya tiene de línea de comandos.

Paso PS1. Consulta de lectura

Con la API iniciada en 5081, ejecuta:

```
$direccion = "http://localhost:5081"
$productos = Invoke-RestMethod -Uri "$direccion/api/productos" -Method Get
$productos | Format-Table
```

La variable productos contendrá la representación deserializada del catálogo, lista para inspeccionarse desde PowerShell. Debes ver Cuaderno y Lápiz cuando el proceso está recién iniciado. Observa la diferencia con curl.exe -i: Invoke-RestMethod resulta cómodo para trabajar con el contenido, mientras que curl.exe -i muestra directamente el estado y las cabeceras.

Paso PS2. Preparar JSON válido

Crea un objeto de datos y conviértelo al formato que espera la API:

```
$solicitud = @{
    nombre = "Agenda"
    precio = 6.40
}
$json = $solicitud | ConvertTo-Json -Compress
$json
```

El resultado debe tener un objeto JSON con nombre y precio. Con este procedimiento evitamos escribir manualmente una secuencia de comillas escapadas dentro de un argumento largo. No enviamos id, porque el identificador pertenece al servidor.

Paso PS3. Crear el producto y observar la respuesta

Ejecuta:

```
$respuesta = Invoke-WebRequest -Uri "$direccion/api/productos" -Method Post -ContentType
"application/json" -Body $json
$respuesta.StatusCode
$respuesta.Headers["Location"]
$respuesta.Content
```

El estado esperado es 201. La cabecera Location identifica la dirección relativa del nuevo recurso. El contenido debe representar el producto creado, incluido el ID que ha asignado la API. Si antes se realizaron otras altas, el identificador concreto puede ser distinto del valor observado en un proceso recién iniciado.

Este método permite estudiar varias piezas de HTTP por separado: estado, cabecera y cuerpo. Después puedes repetir GET del catálogo para comprobar que el nuevo recurso aparece en la memoria del servidor.

Paso PS4. Provocar la regla de rechazo

Prepara una entrada que conserve un nombre pero tenga precio cero:

```
$invalida = @{
    nombre = "Agenda"
    precio = 0
} | ConvertTo-Json -Compress
```

Envía el cuerpo a POST /api/productos y observa el rechazo. Dependiendo del comando y de la versión de PowerShell, una respuesta HTTP no satisfactoria puede presentarse como excepción en vez de devolverse como objeto normal de éxito. No lo interpretes como un error del lenguaje C#: el servidor está comunicando 400 porque los datos incumplen la regla.

Para examinar el código de forma inequívoca también puedes utilizar el comando curl.exe con un archivo JSON:

```
Set-Content -Path .\producto-invalido.json -Value $invalida -Encoding utf8
curl.exe -i -X POST http://localhost:5081/api/productos -H "Content-Type: application/json" --
data-binary "@producto-invalido.json"
```

Comprueba mediante GET que no se ha incorporado un nuevo producto. Esta observación complementa el estado 400 y demuestra que la validación se produjo antes de modificar la colección.

Paso PS5. Diferenciar 404 y ausencia de conexión

Con la API disponible solicita un identificador inexistente usando curl.exe -i. Debes obtener 404 del servidor. Después detén la API y repite GET /api/productos. En ese caso el cliente no recibe un estado HTTP válido del servidor detenido.

El objetivo es comprender que una excepción o un mensaje de error en la terminal no explica por sí solo la causa. Debemos preguntarnos si hubo respuesta HTTP y qué estado contenía. Esa misma clasificación es la que utiliza el componente Blazor cuando distingue rechazo, conexión y deserialización.

Paso PS6. Comparar PowerShell, archivo .http y Blazor

Ahora disponemos de tres clientes distintos para el mismo contrato. PowerShell puede enviar GET y POST; el editor HTTP de Visual Studio permite mantener las solicitudes escritas en un archivo del proyecto; y Blazor integra las operaciones en una interfaz con carga, formularios y mensajes.

Los tres consumidores acceden a la misma API. Por ello la validación del nombre y del precio sigue siendo responsabilidad del servidor, independientemente de que un consumidor presente inputs HTML o que otro envíe un objeto JSON desde PowerShell.

No hace falta incluir los comandos de PowerShell en el código de Blazor: son herramientas para inspeccionar y razonar sobre el comportamiento del servidor. Al terminar, elimina los archivos JSON temporales que hayas creado en tu carpeta de ensayo si no vas a utilizarlos en otros ejercicios.

P4.3 — Una pregunta para comprobar comprensión

Supón que el POST desde PowerShell devuelve 201, pero el formulario Blazor presenta un error. ¿Qué has aprendido? Sabes que la API puede procesar al menos una solicitud válida sin depender del componente. Tu siguiente investigación debería centrarse en el cliente Blazor: dirección base, DTO, servicio tipado o estado de la página.

Ahora supón lo contrario: el formulario Blazor parece mostrar un producto, pero el GET directo a la API no lo incluye. No des por confirmado el alta sin estudiar el estado del servidor. Puede que estés consultando otra instancia o que el componente haya añadido un objeto local sin esperar la confirmación correcta.

Los dos casos demuestran por qué es útil disponer de un consumidor HTTP independiente incluso cuando el objetivo final es construir una interfaz agradable para el usuario.

Ampliación autónoma P4.3.7 — Actualizar y eliminar productos desde cero

Esta ampliación continúa la API construida en P4.3. No es obligatoria durante la jornada de seis horas. Trabajaremos en una copia de la API para incorporar PUT y DELETE sin alterar el recorrido básico de GET y POST. El propósito es comprender contratos, estados HTTP, actualización condicional y eliminación; no crear una segunda aplicación completa.

Paso A1. Preparar una copia y recuperar el estado conocido

Desde la raíz del repositorio copia el proyecto de la API de 4.3 a una carpeta local nueva. Si existe trabajo previo en esa carpeta, elige otra para no sobrescribirlo. No modifiques la API suministrada en 4.1.

```
New-Item -ItemType Directory -Force .work\M04\ampliacion-api | Out-Null
Copy-Item -Path .\M04_APIS_SEGURIDAD\PROYECTO\4.3\CursoWeb.ProductosApi -Destination .
\.work\M04\ampliacion-api\CursoWeb.ProductosApi -Recurse
```

Compila y arranca la copia en 5081 después de detener el servidor anterior. Verifica GET /api/productos y GET /api/productos/1. El estado de referencia contiene Cuaderno y Lápiz. No avances si la consulta inicial no devuelve esos datos: los siguientes pasos compararán sus cambios con ese estado conocido.

Paso A2. Introducir el DTO de actualización

Al final de Program.cs, junto a los records ProductoApi y CrearProductoApi, declara:

```
record ActualizarProductoApi(string Nombre, decimal Precio);
```

¿Por qué no contiene ID? Porque la petición PUT utilizará la ruta /api/productos/{id:int} para seleccionar el recurso que se modifica. El cuerpo tendrá el nombre y el precio nuevo, pero no elegirá otro identificador. Esta decisión mantiene clara la autoridad del servidor sobre la identidad del producto.

Compila. El archivo puede compilar aunque todavía no hayas registrado un endpoint PUT: el record es un tipo de datos, no una ruta. El paso siguiente publicará el comportamiento.

Paso A3. Crear PUT con control de existencia

Antes de app.Run añade:

```
app.MapPut("/api/productos/{id:int}",
    (int id, ActualizarProductoApi entrada) =>
{
    if (string.IsNullOrWhiteSpace(entrada.Nombre) ||
        entrada.Precio <= 0)
        return Results.BadRequest("Nombre y precio deben ser válidos.");

    while (productos.TryGetValue(id, out var anterior))
    {
        var actualizado = new ProductoApi(
            id, entrada.Nombre.Trim(), entrada.Precio);

        if (productos.TryUpdate(id, actualizado, anterior))
            return Results.Ok(actualizado);
    }

    return Results.NotFound();
});
```

El manejador verifica primero los datos. Después intenta leer el recurso por ID. Si no existe, devuelve 404. Si existe, crea una nueva representación que conserva el identificador y utiliza TryUpdate para sustituir el valor si todavía coincide con el que acabamos de observar.

En un laboratorio de memoria, esta forma permite estudiar una condición de actualización sin introducir un DbContext. No afirmamos que sea un sistema general de control de versiones: dos usuarios podrían enviar modificaciones basadas en información antigua. Lo importante aquí es distinguir validación de entrada, existencia de recurso y actualización del diccionario.

Paso A4. Comprobar PUT correcto

Crea actualizar.json:

```
{
  "nombre": "Cuaderno A4",
  "precio": 3.20
}
```

Envía el cuerpo desde PowerShell:

```
curl.exe -i -X PUT http://localhost:5081/api/productos/1 -H "Content-Type: application/json" --
data-binary "@actualizar.json"
```

Debe devolverse 200 junto a un objeto que conserve id igual a 1 y muestre Cuaderno A4 con precio 3,20. A continuación, consulta GET /api/productos/1. La respuesta debe reflejar exactamente la actualización.

La segunda consulta es importante: no basta con comprobar que PUT devolvió un JSON de éxito; queremos demostrar que GET lee la misma colección de memoria y que su estado realmente cambió.

Paso A5. Actualización de un identificador inexistente

Repite PUT con el mismo cuerpo, pero utiliza /api/productos/999. Se espera 404. El JSON es válido, pero la ruta identifica un recurso que no está en el diccionario.

Comprueba que GET /api/productos no ha incorporado un objeto con ID 999. En este contrato PUT no funciona como una creación cuando falta el elemento. La operación de creación sigue siendo POST y es el servidor quien asigna el nuevo ID.

No cambies el código para crear automáticamente un recurso ausente: alterarías el comportamiento que hemos definido para la ampliación y mezclarías actualización con alta.

Paso A6. Regla de precio y nombre

Cambia actualizar.json para que Precio sea cero. Envía PUT sobre el ID 1: debe rechazarse con 400. Consulta GET del producto y confirma que el precio anterior permanece.

Repite la prueba con nombre vacío y precio positivo. También debe devolver 400. Si mezclas nombre vacío y precio cero en una misma prueba, resulta más difícil comprobar qué condición provoca cada rechazo. Por eso ensayamos las reglas individualmente.

El componente Blazor podría impedir que el usuario introduzca esos datos, pero la API debe rechazarlos aunque lleguen desde PowerShell. El fichero HTTP demuestra que la interfaz web no es el único consumidor posible.

Paso A7. Publicar DELETE

Antes de app.Run agrega:

```
app.MapDelete("/api/productos/{id:int}", (int id) =>
    productos.TryRemove(id, out _)
        ? Results.NoContent()
        : Results.NotFound());
```

Compila y reinicia. La petición DELETE recibe un ID desde la ruta, no un DTO con nombre y precio. Cuando encuentra el recurso, TryRemove lo elimina y el manejador devuelve 204 No Content. Si no lo encuentra, devuelve 404.

El estado 204 significa que el servidor no entrega un objeto en el cuerpo de esta respuesta. No debemos intentar deserializarlo como ProductoApi en un cliente. La operación se valida observando el estado HTTP y consultando después la colección.

Paso A8. Comprobar eliminación y repetición

Primero consulta GET /api/productos/2 para confirmar que Lápiz existe. Ejecuta:

```
curl.exe -i -X DELETE http://localhost:5081/api/productos/2
```

La respuesta debe ser 204. Repite GET del ID 2 y observa 404. El recurso ya no existe. Después vuelve a ejecutar DELETE sobre el mismo ID: nuestro contrato devuelve 404 porque la entrada ha sido eliminada.

Este experimento permite razonar sobre idempotencia. El estado final tras dos eliminaciones es el mismo que después de una sola: el producto permanece ausente. No es obligatorio que los dos intentos tengan el mismo código de respuesta para que esa propiedad del efecto se mantenga.

Paso A9. Crear después de eliminar

Con Lápiz eliminado, envía POST /api/productos con Regla y un precio positivo. El servidor asignará un identificador mediante el contador; no se supone que reutilice el ID de un producto eliminado.

Consulta el listado. Debes observar el estado resultante de las operaciones ejecutadas: un producto modificado, otro eliminado y uno nuevo. Si reinicias la API, la colección vuelve a Cuaderno y Lápiz originales. La ampliación no ha cambiado la ausencia deliberada de persistencia.

No introduzcas EF Core para «arreglar» el reinicio en este ejercicio. La finalidad consiste en comprobar el contrato HTTP sobre memoria, con un estado que puede restaurarse fácilmente.

Paso A10. Volver a Blazor sin modificar su contrato

Arranca Blazor 4.2 contra la API ampliada. Abre /catalogo-remoto y ejecuta GET y POST. Ambas operaciones deben continuar funcionando, porque no hemos cambiado sus rutas ni sus DTO. El componente no necesita mostrar todavía botones de editar o eliminar: publicar operaciones nuevas en la API no obliga a que todas las pantallas las utilicen.

Para incorporar una edición remota en una actividad posterior, habría que ampliar IProductosApiService, implementar la llamada HTTP en ProductosApiService y añadir el estado de edición a la UI. No pongas directamente PutAsJsonAsync dentro de CatalogoRemoto.razor si quieres conservar la separación aprendida en P4.2.

Paso A11. Diseñar la operación de actualización para el servicio tipado

En el cliente Blazor podríamos declarar un DTO:

```
public sealed record ActualizarProductoDto(
    string Nombre, decimal Precio);
```

La interfaz añadiría una operación:

```
Task<ProductoDto> ActualizarAsync(
    int id, ActualizarProductoDto solicitud);
```

Y la implementación HTTP contendría:

```
public async Task<ProductoDto> ActualizarAsync(
    int id, ActualizarProductoDto solicitud)
{
    using var respuesta = await cliente.PutAsJsonAsync(
        $"api/productos/{id}", solicitud);
    respuesta.EnsureSuccessStatusCode();

    return await respuesta.Content
        .ReadFromJsonAsync<ProductoDto>()
```

```
        ?? throw new InvalidOperationException(
            "La API no devolvió el producto actualizado.");
}
```

No copies esta operación a una clase sin modificar también su contrato y sus tipos. La actividad de diseño muestra qué archivos cambiarían y qué responsabilidad conservaría cada uno, sin imponer una pantalla adicional durante la jornada presencial.

Paso A12. Diseñar la operación de eliminación para el servicio tipado

La interfaz podría expresar la eliminación mediante Task EliminarAsync(int id). En la implementación:

```
public async Task EliminarAsync(int id)
{
    using var respuesta = await cliente.DeleteAsync(
        $"api/productos/{id}");
    respuesta.EnsureSuccessStatusCode();
}
```

El componente retiraría el producto de su lista después de recibir confirmación. El método no devuelve ProductoDto porque la respuesta 204 del servidor no contiene un nuevo recurso. Si la API indica 404, la operación no se confirma del modo esperado y la UI puede informar al usuario.

Así se conserva el mismo diseño que utilizamos para GET y POST: la interfaz solicita una operación de negocio, la implementación del servicio conoce HTTP y la API valida lo que ocurre sobre su colección.

Paso A13. Restaurar un estado docente claro

Conserva la API mínima y la versión ampliada en lugares identificados. Los pasos obligatorios del punto 4.3 continúan siendo suficientes para avanzar a seguridad básica. PUT y DELETE quedan como extensión autónoma que permite experimentar con nuevas rutas y estados.

Reinicia la API antes de comparar resultados con el catálogo inicial y recuerda que el contador de identificadores no retrocede por eliminar un recurso. Este detalle debe poder justificarse a partir del ciclo de vida del proceso.

Comprobaciones de la ampliación

Petición	Código esperado	Evidencia
PUT /api/productos/1 válido	200	GET devuelve Cuaderno A4
PUT /api/productos/999	404	No se crea el ID 999
PUT con precio cero	400	No cambia el valor guardado
DELETE /api/productos/2	204	GET del ID devuelve 404
Segundo DELETE del mismo ID	404	El producto sigue ausente
POST tras DELETE	201	Servidor asigna nuevo identificador
GET tras reiniciar	200	Vuelven Cuaderno y Lápiz

El objetivo de esta ampliación es poder relacionar método, ruta, validación, respuesta y estado del recurso. No se evalúa por el número de endpoints añadidos, sino por la capacidad del alumno para explicar y ejecutar esos contratos de forma autónoma.

Ampliación autónoma P4.3.8 — Mensajes de validación estructurados desde la API

En el recorrido principal creamos un endpoint POST que rechaza nombre y precio incorrectos mediante un mensaje sencillo. Ahora construiremos **una variante alternativa de ese mismo endpoint** capaz de devolver mensajes separados para cada campo. Esta mejora tiene sentido cuando una interfaz desea informar de manera concreta por qué se rechazó una solicitud.

No cambies los DTO de creación y lectura, la ruta GET ni la estructura del producto creado. La ampliación modifica únicamente el contrato de **respuesta de error** de POST. Se realiza en una copia del proyecto de 4.3 y no forma parte del itinerario presencial obligatorio.

Paso V1. Restaurar un servidor de referencia

Detén todas las API que estén utilizando 5081. Inicia la copia de tu proyecto 4.3 con el POST original todavía activo. Consulta GET /api/productos y comprueba que existen Cuaderno y Lápiz.

Envía un POST con nombre vacío. Debe devolverse 400. Observa que la respuesta de la versión inicial contiene un mensaje general. El servidor ha rechazado la entrada correctamente, pero el cuerpo no identifica los errores por campo.

No modifiques todavía el cliente Blazor. El primer objetivo es comparar dos contratos de error del mismo servidor con herramientas HTTP directas.

Paso V2. Definir qué errores queremos comunicar

Nuestro formulario tiene dos datos de entrada: nombre y precio. El nombre no puede estar vacío ni contener únicamente espacios; el precio debe ser mayor que cero. Queremos que la API distinga ambos motivos de rechazo.

Podemos expresar los errores mediante un diccionario. La clave identifica el campo y el valor contiene uno o varios mensajes. En el ejemplo:

```
var errores = new Dictionary<string, string[]>();

if (string.IsNullOrWhiteSpace(entrada.Nombre))
    errores["nombre"] = ["El nombre es obligatorio."];

if (entrada.Precio <= 0)
    errores["precio"] = ["El precio debe ser positivo."];
```

Si los dos campos son incorrectos, el diccionario tendrá dos claves. Si uno cumple su regla, no aparecerá como error. Esta estructura resulta más útil para un formulario que un mensaje de texto cuyo contenido puede variar libremente.

Paso V3. Sustituir POST, no duplicarlo

En Program.cs comenta el MapPost de creación original y habilita únicamente la alternativa de validación estructurada:

```
app.MapPost("/api/productos", (CrearProductoApi entrada) =>
{
    var errores = new Dictionary<string, string[]>();

    if (string.IsNullOrWhiteSpace(entrada.Nombre))
        errores["nombre"] = ["El nombre es obligatorio."];

    if (entrada.Precio <= 0)
        errores["precio"] = ["El precio debe ser positivo."];

    if (errores.Count > 0)
        return Results.ValidationProblem(errores);

    var id = Interlocked.Increment(ref siguienteId);
    var creado = new ProductoApi(
        id, entrada.Nombre.Trim(), entrada.Precio);
    productos[id] = creado;
    return Results.Created($"/api/productos/{id}", creado);
});
```

Compila y reinicia. No dejes ambos MapPost activos para /api/productos. Si la aplicación contiene dos handlers equivalentes para la misma operación, puedes introducir una selección ambigua de endpoint. El ejercicio consiste en comparar alternativas, no en publicar simultáneamente dos versiones incompatibles.

Paso V4. Probar el nombre incorrecto

Prepara nombre-invalido.json:

```
{
  "nombre": "",
  "precio": 4.50
}
```

Ejecuta:

```
curl.exe -i -X POST http://localhost:5081/api/productos -H "Content-Type: application/json" --
data-binary "@nombre-invalido.json"
```

Comprueba que el estado es 400 y que el cuerpo contiene un objeto de problema. Dentro de errors debe encontrarse una entrada correspondiente a nombre. No debe aparecer un error de precio, porque el valor 4,50 es positivo.

El cuerpo puede incluir propiedades adicionales propias de la representación estándar de problema, como status o title. No dependas del orden textual de las propiedades JSON; comprueba su significado y la presencia de las claves que necesitas.

Paso V5. Probar solamente el precio incorrecto

Sustituye el cuerpo por:

```
{
  "nombre": "Regla",
  "precio": 0
}
```

Vuelve a enviar POST. El servidor debe indicar 400 y un error asociado a precio. El campo nombre no debe incluir un error, porque ahora cumple la regla.

Ejecuta después GET /api/productos. La colección no ha aumentado; el rechazo se ha producido antes de generar el nuevo ID o modificar el diccionario. Una buena respuesta de error también debe estar acompañada del comportamiento correcto sobre el estado.

Paso V6. Probar ambas reglas

Envía:

```
{
  "nombre": "   ",
  "precio": -2
}
```

La API debe encontrar dos problemas y devolverlos bajo las claves nombre y precio. El alumno debería poder relacionar cada mensaje con su condición en el manejador.

Este escenario permite comprobar que la implementación no se limita a rechazar en la primera regla que falla. La variante registra todos los errores que conoce y después produce una sola respuesta de validación.

La colección tampoco debe cambiar. Si GET muestra un producto nuevo después del rechazo, revisa que Interlocked.Increment y la asignación al diccionario ocurren únicamente después de comprobar errores.Count.

Paso V7. Verificar que la creación correcta sigue funcionando

Prepara una entrada válida de Regla por 1,75. El endpoint debe continuar devolviendo 201, un ID nuevo y el producto creado. La ampliación no tenía como objetivo modificar el camino de éxito ni romper el servicio tipado de Blazor.

Consulta GET para confirmar que Regla existe. Después abre /catalogo-remoto desde la aplicación Blazor de 4.2 y comprueba que el catálogo puede leer la lista.

En un POST correcto, ProductosApiService espera un ProductoDto. Como la estructura de éxito sigue siendo la misma, el cliente continúa pudiendo interpretarla. La diferencia de la ampliación se sitúa en las respuestas de error, que la pantalla básica transforma todavía en un mensaje general.

Paso V8. Comparar los dos contratos de error

Revisa la respuesta de la versión original y la de Results.ValidationProblem. Ambas devuelven 400 para los datos inválidos, pero no expresan la información de la misma manera.

La primera versión utiliza un mensaje sencillo que informa del rechazo. La segunda permite distinguir los campos afectados. Si quisiéramos representar esos mensajes al lado de cada input de Blazor, necesitaríamos adaptar ProductosApiService para leer el cuerpo del error y trasladarlo a la presentación de forma explícita.

No supongas que un cliente con EnsureSuccessStatusCode convierte automáticamente el cuerpo de problema a errores de formulario. Ese método informa de que el estado no fue satisfactorio; una experiencia de validación por campo requiere código que interprete el contrato específico de error.

Paso V9. Razonar sobre un JSON mal formado

Hasta ahora enviamos cuerpos JSON válidos, aunque con valores funcionalmente incorrectos. Ahora imagina una entrada que ni siquiera puede convertirse al tipo CrearProductoApi, por ejemplo un número expresado con formato inválido.

En ese caso ASP.NET Core puede rechazar la solicitud durante la lectura del cuerpo, antes de ejecutar nuestra lógica de validación. Por tanto, no debemos afirmar que toda respuesta 400 procede del diccionario errores. Es útil distinguir error de formato, error de enlace a tipo y error de regla de negocio.

La práctica no exige construir manualmente un manejador de excepciones global. Basta comprender que diferentes fases de una petición pueden producir rechazos HTTP y que el contrato del servidor debe permitir investigarlos.

Paso V10. Restaurar la variante adecuada

Al terminar, el alumno puede conservar dos copias de la API: una con el POST inicial y otra con los errores estructurados. Ambas deben compilar, pero solo se ejecuta una implementación concreta de la ruta por proceso.

No modifiques el recorrido obligatorio para presentar esta variante como requisito de la jornada. Se trata de un ejemplo adicional que permite profundizar en diseño de contratos HTTP y reutilizar los conocimientos sobre validación de formularios vistos en el módulo anterior.

Matriz de resultados de P4.3.8

Datos enviados	Estado	Claves de error	Cambio en el catálogo
Nombre vacío, precio 4,50	400	nombre	Ninguno
Nombre Regla, precio cero	400	precio	Ninguno
Nombre espacios, precio negativo	400	nombre y precio	Ninguno
Nombre Regla, precio 1,75	201	Sin errores	Se crea Regla

Esta actividad demuestra que el alumno puede diseñar una respuesta de error clara, comprobarla con HTTP y mantener la compatibilidad del camino de éxito. Es un aprendizaje de contratos, no un ejercicio de añadir páginas.

Resultado del ejercicio

Ahora puedes recorrer una petición desde Blazor hasta la API y viceversa, interpretar sus códigos HTTP y añadir un endpoint nuevo sin modificar rutas existentes. La API continúa funcionando en memoria: al reiniciarla se pierde todo lo creado durante la ejecución.

P4.4 — Demostración local de seguridad

Objetivo

Crea una copia acumulativa de `PROYECTO/4.3/CursoWeb.Blazor` en `PROYECTO/4.4/CursoWeb.Blazor`. Mantén intactas las pantallas de productos y la API auxiliar. La autenticación de demostración **no** constituye Identity ni es apta para producción.

Paso 4.4.1. Configurar el host

En `Program.cs` registra autenticación mediante el esquema `CookiesDemo`, autorización y el estado de autenticación para componentes. Añade los middleware `UseAuthentication()` y `UseAuthorization()`; conserva `UseAntiforgery()`. El código completo está explicado en teoría 4.4.1 y está materializado en el proyecto acumulativo 4.4.

Paso 4.4.2. Identificar la sesión

Abre `Components/Pages/Seguridad.razor`. Examina el `AuthorizeView` que presenta dos resultados alternativos: usuario autenticado o anónimo. La variante `Roles="Formador"` permite observar cómo intervienen los claims.

Paso 4.4.3. Hacer una entrada y salida HTTP

En el formulario POST `/demo/entrar`, utiliza el código local **`curso-demo-44`**. El servidor compara ese código, crea un principal de ejemplo con nombre `AlumnoDemo` y rol `Formador`, y envía una cookie. El token `<AntiforgeryToken />` protege el envío del formulario frente a falsificaciones. Para cerrar la sesión se emplea POST `/demo/salir`, no un enlace GET.

Código del laboratorio: el alumno puede reconstruir el acceso

La demostración no debe limitarse a observar una pantalla preconfigurada. Vamos a localizar y activar los elementos en un orden que permita comprender las dependencias.

Primero, en `Program.cs`, añade los espacios de nombres:

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Antiforgery;
```

Registra los servicios antes de `builder.Build()`:

```
builder.Services.AddAuthentication("CookiesDemo").AddCookie("CookiesDemo", options =>
{
    options.LoginPath = "/seguridad";
    options.Events.OnRedirectToLogin = contexto =>
    {
        if (contexto.Request.Path.StartsWithSegments("/demo/privado"))
        {
            contexto.Response.StatusCode = 401;
            return Task.CompletedTask;
        }
        contexto.Response.Redirect(contexto.RedirectUri);
        return Task.CompletedTask;
```

```
    };
});
builder.Services.AddAuthorization();
builder.Services.AddCascadingAuthenticationState();
```

Coloca `app.UseAuthentication()`, `app.UseAuthorization()` y `app.UseAntiforgery()` antes de mapear los endpoints protegidos.

Después incorpora las rutas de demostración **solo en Development**:

```
if (app.Environment.IsDevelopment())
{
    app.MapGet("/demo/privado",
        () => Results.Ok(new { mensaje = "Recurso protegido" }))
        .RequireAuthorization(p => p.RequireRole("Formador"));

    app.MapPost("/demo/entrar",
        async (HttpContext http, IAntiforgery antiforgery) =>
    {
        if (!await antiforgery.IsRequestValidAsync(http))
            return Results.BadRequest("Token antifalsificación inválido.");

        var formulario = await http.Request.ReadFormAsync();
        if (formulario["codigo"] != "curso-demo-44")
            return Results.Redirect("/seguridad?error=1");

        var claims = new[]
        {
            new Claim(ClaimTypes.Name, "AlumnoDemo"),
            new Claim(ClaimTypes.Role, "Formador")
        };
        var identidad = new ClaimsIdentity(claims, "CookiesDemo");
        await http.SignInAsync(
            "CookiesDemo", new ClaimsPrincipal(identidad));
        return Results.Redirect("/seguridad");
    });

    app.MapPost("/demo/salir",
        async (HttpContext http, IAntiforgery antiforgery) =>
    {
        if (!await antiforgery.IsRequestValidAsync(http))
            return Results.BadRequest("Token antifalsificación inválido.");

        await http.SignOutAsync("CookiesDemo");
        return Results.Redirect("/seguridad");
    });
}
```

Un POST sin token debe devolverse como 400. Esto es intencional: no se debe desactivar la verificación simplemente para simplificar el ejercicio.

Finalmente, crea `Components/Pages/Seguridad.razor`:

```
@page "/seguridad"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@using Microsoft.AspNetCore.Components.Authorization

<h1>Demostración de seguridad</h1>
<AuthorizeView>
    <Authorized>
        <p>Usuario: @context.User.Identity?.Name</p>
```

```
        <AuthorizeView Roles="Formador" Context="rolContexto">
            <Authorized>
                <p>Rol Formador reconocido</p>
            </Authorized>
        </AuthorizeView>
        <form method="post" action="/demo/salir" data-enhance="false">
            <AntiforgeryToken />
            <button type="submit">Cerrar sesión</button>
        </form>
    </Authorized>
    <NotAuthorized>
        <p>Usuario anónimo</p>
        <form method="post" action="/demo/entrar" data-enhance="false">
            <AntiforgeryToken />
            <label>Código:
                <input name="codigo" type="password" />
            </label>
            <button type="submit">Acceder</button>
        </form>
    </NotAuthorized>
</AuthorizeView>
```

Los dos `AuthorizeView` están anidados: el contexto del componente interior tiene nombre explícito para evitar una colisión de parámetros de Razor. Ejecuta la aplicación en entorno Development, que es el único donde existen las rutas de demostración. Las rutas de productos no necesitan autenticación para continuar el ejercicio anterior.

Paso 4.4.4. Verificar una autorización auténtica

Abre `/seguridad` sin sesión. Después intenta consultar directamente `/demo/privado`: el servidor debe impedir el acceso. Inicia sesión en el formulario, consulta el recurso de nuevo y comprueba que ahora devuelve el contenido protegido. Cierra la sesión y confirma que vuelve a denegarlo. Esta comprobación debe hacerse incluso si `AuthorizeView` muestra correctamente el nombre.

Paso 4.4.5. Reto: rol y visibilidad

Comenta temporalmente el bloque visual `AuthorizeView Roles="Formador"` y comprueba que `/demo/privado` sigue protegido. Después modifica **solo en una copia de ensayo** la regla de autorización del endpoint para exigir `OtroRol`; aunque el usuario de prueba continúe autenticado, el servidor debe denegar acceso. Restablece `Formador`.

Ampliación P4.4 — Recorrido completo de la sesión

El propósito de este laboratorio es observar una sesión autenticada y comprobar un recurso protegido **del lado del servidor**. La pantalla de seguridad sirve para mostrar el estado, pero el criterio de autorización de /demo/privado no se sustituye por esconder el enlace.

Antes de iniciar, confirma que la aplicación está ejecutándose en **Development**. La versión del curso registra los endpoints de demostración únicamente en ese entorno. No actives estas rutas en Production para resolver un fallo de ejecución, porque el código de acceso es fijo y no constituye una credencial de producción.

Trabajaremos con una sesión limpia del navegador. Cierra las pestañas abiertas de la aplicación o usa una ventana privada para evitar que una cookie creada en una ejecución anterior modifique el estado inicial. Abre la página /seguridad y observa su aspecto sin iniciar sesión. El componente debe presentar la rama anónima de AuthorizeView y el formulario para introducir el código.

Escenario 4.4.A — Usuario anónimo: dos observaciones distintas

Primero mira la página. El texto de usuario anónimo demuestra que la presentación conoce un estado de autenticación sin identidad reconocida. Todavía no demuestra que el servidor proteja un recurso.

Segundo, introduce manualmente en la barra de direcciones la ruta /demo/privado. El servidor debe impedir que obtengas el objeto JSON de éxito. En esta demostración está configurada una respuesta 401 para el recurso anónimo. Anota la diferencia entre una página que decide qué dibujar y un endpoint que decide si entrega datos.

El resultado esperado no consiste en que el enlace desaparezca. Si el recurso devolviera 200 a una petición anónima, existiría una falla real de autorización incluso aunque la interfaz ocultara la posibilidad de navegar hasta él.

Escenario 4.4.B — Credencial de laboratorio incorrecta

Vuelve a /seguridad y escribe un código diferente de curso-demo-44. Envía el formulario y comprueba que no se reconoce AlumnoDemo ni aparece la rama del rol Formador. El endpoint de acceso no debe emitir una sesión válida en este caso.

Observa qué hace el navegador después del POST. La implementación redirige nuevamente a la página de demostración, añadiendo un parámetro de error en la URL. El parámetro permite distinguir un intento incorrecto del acceso inicial si decidimos mostrar esa información, pero no sustituye la comprobación del servidor.

No modifiques el código para admitir cualquier cadena simplemente porque el alumno no recuerda el valor de ejemplo. Ese cambio anularía precisamente la condición que queremos observar: que el servidor decide cuándo crea la identidad.

Escenario 4.4.C — Inicio de sesión válido

Introduce el código de laboratorio correcto y envía el formulario. El servidor valida el token antifalsificación, lee el campo codigo y comprueba su valor. Después crea un principal con nombre AlumnoDemo y claim de rol Formador.

La redirección completa permite observar una nueva navegación. La pantalla debe representar la rama Authorized y mostrar el nombre. El bloque anidado AuthorizeView Roles="Formador" debe reconocer el rol. En el navegador, el conjunto de cookies correspondiente al host habrá cambiado después de iniciar sesión.

No hace falta copiar el valor de la cookie ni colocarlo en el Markdown: es información temporal de la sesión que no aporta nada al aprendizaje y no debe circular fuera del navegador.

Escenario 4.4.D — La política del servidor permite el recurso

Con la sesión iniciada, abre /demo/privado. Esta vez el servidor debe devolver el JSON del recurso protegido. La diferencia respecto a 4.4.A no es la URL, que no ha cambiado, sino la identidad que el servidor reconoce y su claim de rol.

Vuelve a /seguridad y observa que el componente también reconoce Formador. Son dos observaciones que coinciden porque se apoyan en el mismo estado de autenticación, pero su responsabilidad es diferente: la página controla contenido representado y el endpoint controla acceso a información.

Escenario 4.4.E — Salida mediante POST

Pulsa el botón Cerrar sesión de la página. La acción envía un POST a /demo/salir, el servidor valida el token antifalsificación, ejecuta SignOutAsync y redirige. La página debe volver a presentar la rama anónima.

Comprueba de nuevo /demo/privado. El servidor debe rechazar el acceso. Esta segunda consulta es imprescindible: si solo miráramos que el texto visual volvió a Usuario anónimo, no estaríamos demostrando que la credencial dejó de autorizar recursos.

Escenario 4.4.F — POST sin antiforgery

Desde una herramienta HTTP que no adjunte el token antifalsificación, intenta enviar POST al endpoint /demo/entrar con un código correcto. El resultado esperado es un rechazo 400, no una sesión autenticada.

La clave del caso es que conocer el código de laboratorio no dispensa al cliente de la comprobación antifalsificación del formulario. Ese mecanismo evita ciertas solicitudes de cambio de estado originadas en sitios ajenos. No es una contraseña alternativa ni una forma de decidir qué rol debe tener una persona.

Si se obtiene una cookie válida sin token, revisa si la implementación activa exige realmente IsRequestValidAsync en el endpoint, en lugar de confiar únicamente en que el componente Razor haya mostrado AntiforgeryToken.

Escenario 4.4.G — Rol correcto y rol equivocado

Vuelve a iniciar sesión con el código de laboratorio. En una copia temporal de Program.cs cambia la política del endpoint /demo/privado para que exija OtroRol en vez de Formador. Compila y reinicia la aplicación, iniciando una sesión nueva con el principal que sigue teniendo Formador.

La pantalla reconocerá que existe un usuario autenticado, pero el servidor no debe entregar el recurso que ahora exige OtroRol. La observación esperada es una denegación para una identidad conocida, no la desaparición de la cookie.

Restablece la política Formador y repite la operación. Esta variante enseña que autenticación y autorización son evaluaciones distintas. No debe quedar activada como estado de referencia al terminar el laboratorio.

Escenario 4.4.H — Ocultar la rama visual no protege recursos

Comenta temporalmente únicamente el bloque AuthorizeView Roles="Formador" de Seguridad.razor. No alteres el código del endpoint ni la cookie. Al iniciar sesión puede desaparecer el mensaje que anunciaba el rol, pero el recurso /demo/privado sigue verificando RequireRole("Formador").

Si el servidor responde de forma distinta solo porque hemos cambiado una rama visual, entonces hemos introducido un acoplamiento indebido que no existe en el ejercicio original. Esa observación resume la idea más importante de 4.4: las restricciones efectivas se verifican donde están los recursos.

Escenario 4.4.I — El entorno también forma parte del contrato de demostración

Ejecuta, solo como comprobación controlada, la aplicación en un entorno diferente de Development. Las rutas de acceso de la demostración están deliberadamente restringidas y no deben quedar publicadas como mecanismo real de seguridad.

No interpretes que dejan de estar disponibles como un error que deba arreglarse eliminando el condicional de entorno. La restricción expresa una decisión de seguridad: el código fijo sirve para la sesión formativa, no para desplegar una aplicación pública.

Al volver al estado normal, restablece Development y no cambies configuraciones globales de la máquina que puedan afectar a otros proyectos del curso.

P4.4 — Qué observar en cada capa

Elemento	Antes de entrar	Después de entrar	Después de salir
Página /seguridad	Rama anónima	Usuario AlumnoDemo	Rama anónima
Claim de rol	Sin Formador	Formador	Sin Formador
Recurso /demo/privado	Denegado	Permitido	Denegado
Cookie local reconocida	No	Sí	No
API /api/productos separada	Pública en el laboratorio	Sin cambio	Sin cambio

El último renglón evita una conclusión falsa. La API auxiliar del ejercicio se ejecuta en otro proceso y conserva deliberadamente sus endpoints públicos. El inicio de sesión en Blazor no convierte automáticamente la API en un servidor que comparta identidad.

P4.4 — Dificultades frecuentes y correcciones explicadas

El formulario se ve, pero tras enviar no cambia el nombre del usuario. Confirma que se envía mediante POST normal y que data-enhance="false" evita una navegación mejorada incompatible con el objetivo de observar la nueva cookie. Comprueba que el código es correcto y que se ha incluido AntiforgeryToken. Si el servidor devuelve 400, investiga primero la validación del token.

La compilación informa sobre Authorized y context duplicado. Has anidado dos componentes AuthorizeView que usan el mismo identificador de plantilla. Asigna Context="rolContexto" al interior para distinguir sus parámetros. No es un error de rol: ocurre en la compilación Razor.

La página reconoce al usuario, pero /demo/privado sigue denegando. Comprueba qué rol exige la política y cuál está incluido en el principal. Si cambiaste temporalmente a OtroRol, restaura Formador. Comprueba también que solicitas el recurso del mismo host que emitió la cookie.

La página local /productos falla después de añadir autenticación. Revisa que no hayas eliminado las dependencias del módulo 3 ni cambiado sin necesidad su ruta. La demostración de seguridad no exige reescribir los servicios de memoria.

La API de productos falla, aunque /seguridad funciona. Comprueba los procesos por separado. Una pantalla de autenticación visible en Blazor no demuestra que la API auxiliar esté iniciada ni que la dirección de ProductosApi:BaseUrl sea correcta.

No aparece /demo/entrar. Revisa el entorno en que se ejecuta la aplicación. Si no es Development, la ruta está ausente deliberadamente. No la habilites en un entorno público por conveniencia.

El recurso anónimo redirige en vez de mostrar un JSON 401. Algunas configuraciones de cookies redirigen al formulario de acceso por defecto. El ejemplo incluye una excepción para la ruta HTTP del laboratorio que responde 401, facilitando la comprobación. Para una aplicación real sería necesario diseñar deliberadamente el comportamiento de cada tipo de cliente.

P4.4 — Recuperar el estado limpio del proyecto

Después de ensayar roles alternativos, formularios incorrectos y comentarios de presentación, el proyecto debe regresar a la configuración de referencia. Deja la política del recurso exigiendo Formador; la identidad de prueba incluye el mismo rol; el formulario de acceso conserva su token; la salida se efectúa con POST; y las rutas de laboratorio siguen restringidas a Development.

Inicia una sesión limpia y ejecuta la secuencia mínima: anónimo denegado, acceso correcto, usuario y rol visibles, recurso permitido, salida y recurso denegado de nuevo. Una explicación oral de cada transición aporta más evidencia de comprensión que una captura aislada de la página tras iniciar sesión.

P4.4 — Preguntas de interpretación con respuesta razonada

¿Por qué AuthorizeView no sustituye RequireAuthorization? Porque AuthorizeView condiciona la representación del componente, mientras que RequireAuthorization se comprueba al ejecutar el recurso del servidor. Un cliente distinto de la interfaz podría solicitar directamente el endpoint.

¿Por qué usamos POST para cerrar sesión? Porque cambia el estado de autenticación. Un enlace GET no debería emplearse para una operación de cambio de estado, especialmente cuando está asociada a una cookie que un navegador puede enviar automáticamente.

¿Por qué el token antifalsificación no es una contraseña? Porque no identifica a la persona ni determina sus permisos. Su finalidad es comprobar la legitimidad del envío de una solicitud en el contexto de un formulario y una sesión.

¿Por qué no usamos aquí Identity completo? Porque el objetivo del módulo es comprender autenticación, autorización, cookies y estado visual, sin volver a impartir una infraestructura de gestión de usuarios. El ejemplo es controlado y no debe utilizarse para producción.

¿Por qué hay que probar el endpoint después de cerrar sesión? Porque la pantalla puede cambiar por razones visuales. El criterio de seguridad consiste en que el servidor deje de entregar un recurso cuando la identidad ya no satisface la política.

Ampliación transversal de M04 — Recuperar una aplicación con cuatro fallos aislados

Esta actividad combina los conceptos de los cuatro puntos sin añadir nuevas tecnologías ni convertir la API de memoria en un producto empresarial. Se realiza en una **copia de ensayo del estado 4.4**, nunca sobre el proyecto de referencia sin conservar el estado original. El propósito es que el alumno aprenda a relacionar una observación con el límite del sistema responsable.

El docente puede presentar las incidencias una a una, o el alumno puede provocarlas y corregirlas siguiendo los escenarios. No se exige resolverlas todas durante el tiempo presencial; forman parte

del material de referencia y consolidación. Para cada caso trabajaremos con un único fallo intencionado, de modo que su causa sea identificable.

Preparación T0. Crear una copia limpia y conocer los dos procesos

Localiza el proyecto Blazor 4.4 y la API del ejercicio 4.3. Conserva una copia de ambos estados tal como están versionados. La API se ejecutará en 5081 y Blazor en un puerto distinto. Antes de cambiar nada, confirma con una petición directa que el servidor devuelve Cuaderno y Lápiz.

Ahora abre /catalogo-remoto. Debes observar los mismos productos; esa es la primera referencia visual. A continuación abre /seguridad, comprueba que aparece la rama anónima y solicita directamente /demo/privado para observar la denegación de acceso.

No continúes con las incidencias si el punto de partida ya presenta errores. Resolver un problema introducido por nosotros requiere saber cómo se comportaba la aplicación antes.

Incidencia T1. Blazor funciona, pero la API parece haber desaparecido

Detén solo el proceso API. Mantén abierto Blazor e intenta consultar /catalogo-remoto. La página deberá seguir existiendo, aunque muestre un error de comunicación. Si la navegación hasta la página también falla, quizás has detenido el proceso equivocado.

A continuación, deja la API iniciada en otro puerto, por ejemplo 5088, y mantén ProductosApi:BaseUrl apuntando a 5081. El síntoma desde Blazor puede ser parecido, pero ahora la API sí existe: simplemente no está escuchando en la dirección que utiliza el cliente.

Diagnostica en este orden. Comprueba con el navegador o el archivo .http qué puerto responde; después abre appsettings.json de Blazor; finalmente verifica la URL base que se utiliza en el registro de HttpClient. Si se ha movido la API a 5088, cambia solo la configuración del cliente y vuelve a ejecutar.

No reescribas ProductoDto ni el componente Razor para arreglar un puerto. Son partes del sistema que no han cambiado. El criterio de corrección es que GET y POST vuelvan a funcionar sin modificar el contrato de datos.

Incidencia T2. La página no obtiene siquiera un servicio

Restaura 5081. En una copia de ensayo de Program.cs comenta temporalmente el registro de IProductosApiService. Ejecuta el proyecto y navega a /catalogo-remoto. El componente solicita una dependencia que el contenedor ya no sabe proporcionar.

Compara esta incidencia con T1. En T1 la resolución de dependencias era correcta, pero la petición HTTP no alcanzaba su destino. En T2 la aplicación no llega a ejecutar la operación remota porque no puede construir el colaborador del componente.

Restaura el registro tipado y compila:

```
builder.Services.AddHttpClient<IProductosApiService, ProductosApiService>(
    cliente =>
    {
        cliente.BaseAddress = new Uri(
            builder.Configuration["ProductosApi:BaseUrl"]
            ?? "http://localhost:5081/");
        cliente.Timeout = TimeSpan.FromSeconds(10);
    });
```

Comprueba que /productos —la página local procedente de M03— sigue disponible. No borres IProductosMemoriaService mientras estás corrigiendo el consumo remoto: hacerlo introduciría una regresión independiente del fallo que queremos estudiar.

Incidencia T3. El servidor responde 200, pero los datos no pueden leerse

Con ambos procesos iniciados, observa la respuesta directa de GET /api/productos. Debe ser un arreglo JSON. Imagina ahora que sustituyéramos su cuerpo por el objeto de resumen que contiene cantidad y precioTotal. La ruta podría seguir devolviendo 200, pero el servicio tipado que espera List no podría interpretar la nueva forma.

Para observar un fallo real sin destruir el contrato de manera permanente, utiliza una copia de ensayo del Program.cs de la API y sustituye temporalmente el GET del listado por:

```
app.MapGet("/api/productos", () =>
    Results.Ok(new { cantidad = productos.Count }));
```

No registres esta ruta junto a la versión original. Sustituye una definición por otra, compila y reinicia. Consulta desde el navegador: obtendrás un objeto JSON. Después abre /catalogo-remoto y observa el fallo de conversión, porque el cliente sigue esperando una lista.

Este problema no se resuelve aumentando el timeout ni cambiando el puerto. El servidor respondió a tiempo, pero cambió la forma de los datos. La solución de la actividad es restaurar GET /api/productos como lista, manteniendo cualquier nuevo resumen en su ruta independiente.

Incidencia T4. La interfaz oculta el rol, pero el recurso sigue protegido

Restaura la API y comprueba que el catálogo funciona. Ahora entra en /seguridad con el código de laboratorio en Development y consulta /demo/privado. Debería permitir el acceso a la identidad con rol Formador.

Comenta temporalmente la representación visual del rol en Seguridad.razor, sin alterar la política del endpoint. La página puede dejar de anunciar el rol, pero el servidor conserva la exigencia de Formador. Este resultado demuestra que la interfaz no decide por sí misma si entrega información protegida.

Después cambia solo la política del endpoint para exigir OtroRol. El usuario sigue autenticado, pero deja de satisfacer el requisito. Restaura Formador al terminar y realiza una salida mediante POST. Comprueba de nuevo que el recurso deniega el acceso anónimo.

El criterio de corrección no es que el formulario parezca convincente: es observar que el servidor toma la decisión de autorización y que la sesión deja de permitir el recurso después del cierre.

Taller integrador — Secuencia de recuperación de extremo a extremo

Cuando hayas revertido los cuatro defectos, realiza una secuencia completa con el estado de referencia:

1. Inicia la API del laboratorio en 5081 y consulta GET /api/productos desde una herramienta HTTP. Debes recibir Cuaderno y Lápiz.
2. Inicia Blazor y abre /catalogo-remoto. Debes recibir la misma colección a través de IProductosApiService.
3. Crea Regla por 1,75 desde el formulario y observa el ID asignado por el servidor.

4. Consulta GET /api/productos directamente y comprueba que Regla existe en memoria.
5. Envía una petición POST con precio cero y comprueba que el servidor la rechaza sin crear otro recurso.
6. Reinicia la API y observa que regresa al estado inicial de dos productos.
7. Abre /seguridad en Development y comprueba que un usuario anónimo no recibe /demo/privado.
8. Entra con el código de laboratorio, comprueba el rol Formador y solicita de nuevo el recurso privado.
9. Cierra la sesión mediante el formulario POST y vuelve a comprobar la denegación del recurso.

Esta secuencia no exige que el alumno presente una captura por cada acción. Su valor está en poder explicar por qué cada resultado corresponde a una decisión determinada de interfaz, servicio, API o autorización del servidor.

Taller integrador — Comparación de diagnósticos

Síntoma observado	Primera comprobación útil	Límite responsable
Blazor abre pero no encuentra catálogo	Consultar API directamente y revisar puerto	Comunicación
Error al activar CatalogoRemoto	Revisar registro IProductosApiService	Contenedor DI
HTTP 200 con error de JSON	Comparar cuerpo con ProductoDto	Contrato
POST inválido crea un producto	Revisar validación en servidor	Reglas API
Dos productos creados por dos pulsaciones	Revisar estados de envío y semántica POST	UI y operación
Resumen incompatible con catálogo	Revisar rutas diferenciadas	Diseño de API
Formulario muestra anónimo tras entrar	Revisar POST, token y cookie	Autenticación
Usuario identificado sin permiso	Revisar claims y RequireRole	Autorización
Link oculto pero endpoint accesible	Probar URL directamente	Protección del recurso
Nuevo proceso de API muestra solo dos productos	Revisar almacenamiento de memoria	Ciclo de vida del proceso

Taller integrador — Reflexión sobre el diseño final

El componente CatalogoRemoto presenta datos y recibe interacciones; IProductosApiService declara operaciones; ProductosApiService conoce HTTP y el contrato remoto; la API valida y modifica su colección; y el servidor Blazor aplica autorización al endpoint de demostración. La corrección funcional depende de que estos límites cooperen, no de que una única clase reúna toda la lógica.

El alumno debería poder construir una pequeña API compatible con el cliente, observar estados de error sin confundirlos y explicar por qué una vista que oculta un control no asegura un recurso. Esa es la competencia integrada que M04 aporta al proyecto del curso.

No introducimos en esta actividad base de datos, un proveedor de identidad completo ni despliegue público. Esos ámbitos modificarían el alcance de seis horas que hemos acordado. El

material adicional proporciona profundidad, variantes y diagnóstico, pero no exige que cada escenario se imparta íntegramente durante el tiempo presencial.

Ampliación autónoma P4.4 — Probar la diferencia entre identidad reconocida y permiso insuficiente

El recorrido de seguridad de P4.4 demuestra que una persona anónima no puede abrir /demo/privado y que la identidad de laboratorio con rol Formador sí puede. Hay un tercer caso relevante: una identidad autenticada que no posee el rol requerido. En esta ampliación construiremos deliberadamente ese estado en una **copia del proyecto 4.4**.

No cambiaremos la API de productos. Los endpoints que vamos a comprobar pertenecen al host Blazor y permanecen restringidos a Development. No publicaremos un nuevo sistema de acceso real ni añadiremos cuentas persistentes.

Paso R1. Preparar una copia de seguridad del escenario

Abre el proyecto Blazor de 4.4 y conserva una copia de su Program.cs. Compila y ejecuta la aplicación en entorno Development. Desde una ventana privada del navegador, abre /seguridad y comprueba que se muestra Usuario anónimo.

Es importante trabajar con una sesión limpia. Si el navegador conserva una cookie anterior, podrías estar comprobando un principal con rol Formador aunque todavía no hayas ejecutado el formulario durante esta prueba.

Consulta directamente /demo/privado sin iniciar sesión. El servidor debe impedirte recibir el objeto JSON protegido. En el laboratorio el flujo para un usuario anónimo está configurado para responder 401. Esta es la primera referencia.

Paso R2. Definir qué identidad alternativa necesitamos

Una identidad autenticada se construye a partir de claims. En la versión base creamos un principal con nombre AlumnoDemo y rol Formador. Para examinar el caso de acceso denegado por rol necesitamos construir otra identidad, también autenticada, pero con un rol distinto, por ejemplo Lector.

No es necesario añadir una tabla de usuarios. Vamos a aceptar un segundo código fijo **solo en esta copia de desarrollo**, con la finalidad de generar el principal alternativo. El código de Formador seguirá siendo curso-demo-44; el código lector será curso-demo-lector.

Estas cadenas existen únicamente para el laboratorio. No deben convertirse en credenciales de una aplicación publicada.

Paso R3. Modificar la comprobación del formulario en una copia

Dentro de la ruta POST /demo/entrar localiza la comprobación del campo codigo. En la versión básica se acepta una sola cadena. Sustituye esa parte por:

```
var codigo = form["codigo"].ToString();
var rol = codigo switch
{
    "curso-demo-44" => "Formador",
    "curso-demo-lector" => "Lector",
    _ => null
};
```

```
if (rol is null)
    return Results.Redirect("/seguridad?error=1");
```

Más adelante, cuando prepares los claims, conserva ClaimTypes.Name y sustituye el valor fijo del rol por la variable rol:

```
var claims = new[]
{
    new Claim(ClaimTypes.Name, "AlumnoDemo"),
    new Claim(ClaimTypes.Role, rol)
};
```

No modifiques el esquema CookiesDemo ni el registro general de autenticación. El objetivo es cambiar el contenido de la identidad para la sesión de ensayo, no introducir un nuevo proveedor o una cookie diferente.

Compila. Si obtienes un error de variable local repetida, asegúrate de haber sustituido el bloque anterior y no de haber declarado otra variable claims junto a la original.

Paso R4. Configurar un rechazo HTTP observable para falta de permiso

En las opciones AddCookie ya existe un evento OnRedirectToLogin utilizado para que las peticiones anónimas a /demo/privado reciban 401. Añade también el tratamiento de acceso denegado para ese endpoint:

```
options.Events.OnRedirectToAccessDenied = contexto =>
{
    if (contexto.Request.Path.StartsWithSegments("/demo/privado"))
    {
        contexto.Response.StatusCode = 403;
        return Task.CompletedTask;
    }

    contexto.Response.Redirect(contexto.RedirectUri);
    return Task.CompletedTask;
};
```

La primera condición no sustituye la segunda. OnRedirectToLogin se ocupa del caso de autenticación ausente, mientras que OnRedirectToAccessDenied permite expresar falta de autorización de un principal ya reconocido. Los códigos 401 y 403 corresponden a situaciones distintas.

La política del recurso debe permanecer:

```
app.MapGet("/demo/privado", () =>
    Results.Ok(new { mensaje = "Recurso protegido" }))
    .RequireAuthorization(p => p.RequireRole("Formador"));
```

No cambies RequireRole por Lector, porque perderíamos la referencia de permiso que queremos comprobar.

Paso R5. Comprobar el caso anónimo

Con una nueva sesión privada, solicita /demo/privado antes de enviar ningún formulario. El resultado esperado sigue siendo 401.

El navegador podría mostrar una página de error con información resumida o vacía; eso no significa que el endpoint haya entregado su contenido protegido. Para observar el código con precisión puedes utilizar las herramientas de desarrollo del navegador y examinar la respuesta en la pestaña Red o utilizar un cliente HTTP.

No envíes una cookie inventada ni copies valores de sesión a otro documento. El navegador gestionará la cookie que emita nuestro host de desarrollo.

Paso R6. Autenticarse como Lector

En /seguridad escribe curso-demo-lector y envía el formulario. El servidor debe aceptar el código alternativo, emitir una cookie y devolver una navegación que reconozca al usuario. El nombre AlumnoDemo puede continuar apareciendo, pero el bloque AuthorizeView Roles="Formador" no debe mostrar el mensaje de rol Formador.

Esto demuestra que una identidad reconocida no satisface automáticamente todas las reglas. El usuario Lector está autenticado, pero no posee el claim de rol requerido por el endpoint.

Consulta ahora /demo/privado. El servidor debe devolver 403 y no el objeto JSON de éxito. Si observas 200, revisa el claim de rol y la política RequireRole. Si observas 401, investiga si el navegador ha enviado la cookie de sesión correcta o si la autenticación no llegó a establecerse.

Paso R7. Probar el rol Formador sin cambiar el endpoint

Cierra la sesión de Lector mediante el formulario POST de salida. Vuelve a /seguridad e inicia sesión con curso-demo-44. Ahora AuthorizeView debe reconocer Formador y /demo/privado debe devolver 200 con su JSON.

No modifiques el endpoint entre ambas pruebas. El hecho de que la misma ruta dé 403 para Lector y 200 para Formador constituye una observación clara de que la autorización depende de la política aplicada sobre el principal.

Paso R8. Probar credenciales incorrectas como escenario independiente

Cierra la sesión e introduce una cadena distinta de los dos códigos admitidos. El servidor debe redirigir al formulario sin crear una cookie autenticada. La siguiente consulta a /demo/privado debe permanecer denegada como petición anónima.

No confundas esta situación con Lector: en el caso de código incorrecto no se ha reconocido una identidad; en el caso de Lector sí se ha creado una sesión, pero sin el rol necesario para el recurso.

La distinción se reconoce mejor si las pruebas se ejecutan una a una y con sesiones limpias.

Paso R9. Verificar un POST sin token antifalsificación

Sin sesión iniciada, intenta enviar POST /demo/entrar con un cliente HTTP que no adjunte el token del formulario. Incluso si utilizas un código reconocido, el servidor debe rechazar la operación por falta de token válido.

Este caso no demuestra que los roles funcionen, porque la solicitud puede rechazarse antes de construir cualquier identidad. Su finalidad es comprobar una protección independiente, relacionada con el envío de formularios y las cookies del navegador.

Una prueba de autorización no sustituye a una prueba de antiforgery, y viceversa. Ambas se necesitan para comprender el recorrido completo.

Paso R10. Restaurar el estado de referencia

Cuando termines, restaura Program.cs para aceptar solo el código original del laboratorio y devolver únicamente el rol Formador. Conserva la política RequireRole y la comprobación antifalsificación. Las rutas continúan registradas exclusivamente en Development.

La variación Lector no debe quedar activada inadvertidamente en el proyecto de referencia del curso. Puedes guardar la copia de ensayo como ejercicio autónomo, pero el estado oficial de P4.4 debe seguir siendo sencillo y reproducible.

Pruebas comparadas de identidad y autorización

Estado del usuario	Cookie reconocida	Rol	/demo/privado
Anónimo	No	Ninguno	401
Código incorrecto	No	Ninguno	401
Código lector de ensayo	Sí	Lector	403
Código original del laboratorio	Sí	Formador	200
Después de cerrar sesión	No	Ninguno	401

Esta tabla expresa cinco estados distintos del recorrido de seguridad. La prueba es válida cuando el alumno puede reproducirlos y explicar el motivo de cada respuesta, no simplemente cuando ve el texto de un rol en la pantalla.

Interpretación final de la ampliación

El ejemplo conserva una idea principal: autenticación y autorización son decisiones diferentes. Un claim de nombre permite representar una identidad reconocida; un claim de rol permite comprobar una política concreta. Las respuestas 401 y 403 pueden comunicar que falta autenticación o que faltan permisos, aunque la configuración de cookies de otros proyectos pueda decidir redirigir a páginas HTML.

Por ello, una interfaz que oculta el enlace a un recurso no sustituye la exigencia de rol dentro del endpoint. La seguridad del recurso reside en el servidor y debe poder probarse independientemente del marcado Razor.

Cierre

Hemos observado autenticación, autorización y representación visual de la identidad sin rehacer cursos de Web API o Identity. Nunca reutilices la credencial fija de laboratorio en un despliegue público.